Applied Soft Computing Journal 184 (2025) 113712

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Graph-based deep reinforcement learning for dynamic scheduling of flexible job-shop considering worker fatigue and multi-skill factors

Yiwen Hu, Zequn Zhang *, Jie Chen, Dunbing Tang, Qixiang Cai

College of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, PR China

HIGHLIGHTS

- A DRC-DFJSP model is developed to address dynamic events, while considering factors like fatigue accumulation and multi-skilled worker proficiency.
- Two objective functions are combined to define the reward function and state information for the agent, enabling the learning of scheduling policies.
- A novel heterogeneous graph neural network converts the DRC-DFJSP disjunctive graph structure into state features for DRL agent decision-making.

ARTICLE INFO

Keywords:
Dual-resource constrained
Deep reinforcement learning
Dynamic scheduling
Graph neural network
Worker fatigue

ABSTRACT

In recent years, the dynamic scheduling problem of flexible job shops (DFJSP) has garnered significant attention. However, most DFJSP studies focus solely on machine constraints, often overlooking the crucial factor of worker constraints. As an essential resource within the production process, the efficient utilization of workers can substantially enhance production efficiency. Consequently, the scheduling problem in dual-resource constrained (DRC) production systems, where operation times fluctuate according to worker skill levels and fatigue, is investigated. Considering new job arrivals as dynamic events, this study comprehensively integrates worker skill and fatigue factors, with the goal of minimizing total cost and makespan by developing a dual-resource constrained dynamic flexible job-shop scheduling (DRC-DFJSP) optimization model. To enable real-time model resolution in dynamic environments, an end-to-end deep reinforcement learning (DRL) scheduling method is introduced. The decision-making process is formulated as a Markov decision process (MDP) and guided by a reward mechanism tailored to optimization objectives. An enhanced proximal policy optimization (PPO) algorithm, combined with an adaptive clipping mechanism and a prioritized experience replay buffer, is applied to handle operation ordering, machine assignment, and worker allocation within the DRC-DFJSP framework. To further enhance decision-making capabilities, an attention-based graph neural network feature extraction method is incorporated to capture the intricate connections between operations, machines, and workers, resulting in a more precise characterization of the workshop state. Numerical experiments and case studies demonstrate that the proposed scheduling method surpasses existing strategies in dynamic environments that account for worker skill and fatigue levels.

1. Introduction

The flexible job-shop scheduling problem (FJSP) is essential in contemporary manufacturing. In the classical FJSP, operations are sequenced to optimize specific metrics, with each operation allocated to one machine selected from a pool of eligible options [1]. In recent years, with the advancement of high-end manufacturing domains such as equipment manufacturing, automotive production, and aerospace, human factors and workforce scheduling have gained widespread attention due to the need for workers to control key operations in the production of precision products. This has posed greater challenges to the formulation and resolution of job-shop scheduling problems. A workshop scheduling scenario where both machines and workers are constrained in quantity is referred to as dual-resource constrained (DRC) [2]. Differences in worker skill levels and efficiency introduce flexibility in human labor. The scheduling problem that accounts for both machine and worker constraints, as well as flexibility, is called the dual-resource constrained flexible job-shop scheduling problem (DRC-FJSP).

* Corresponding author.
E-mail address: zhjj370@nuaa.edu.cn (Z. Zhang).

https://doi.org/10.1016/j.asoc.2025.113712
Received 10 January 2025; Received in revised form 24 July 2025; Accepted 30 July 2025
Available online 7 August 2025
1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

DRC-FJSP connects theoretical scheduling models with practical production environments. However, most studies on DRC-FJSP do not account for dynamic events, whereas real-world manufacturing systems often encounter uncertainties such as job insertions, and random job arrivals frequently occur, causing deviations between planning and execution and rendering original schedules ineffective. Therefore, studying the dual-resource constrained dynamic flexible job-shop scheduling problem (DRC-DFJSP) better aligns with real production environments.

Due to harsh working environments, increased automation, and rising labor costs, real-world workshops generally employ fewer workers compared to the number of available machines [3]. To meet production demands, companies often provide cross-training to workers, turning them into multi-skilled employees. In general, multi-skilled workers differ in terms of skill combinations and proficiency levels. When assigning tasks, incorporating variations in worker skills and proficiency levels can effectively reduce production costs while improving efficiency and product quality [4]. However, worker fatigue is inevitable, and prolonged accumulation of fatigue can harm workers' health, reduce their operational efficiency, degrade production system performance [5], and even lead to safety incidents and economic losses. In summary, both worker proficiency and fatigue are crucial to DRC production systems; however, research that simultaneously considers both factors remains limited. This gap between scheduling models and actual production results in a lack of practicality in scheduling solutions. Therefore, combined with the aforementioned dynamic events, this paper focuses on studying DRC-DFJSP while considering worker fatigue and proficiency, which holds significant practical relevance for guiding real-world production.

Currently, there are two main approaches to solving dynamic scheduling problems: predictive-reactive scheduling (rescheduling) [6] and completely reactive scheduling (real-time scheduling) [7]. Predictive-reactive scheduling addresses dynamic problems by transforming them into static instances, followed by metaheuristic-based generation of scheduling plans. Although predictive-reactive scheduling can produce high-quality schedules, it requires significant time to search for new schedules whenever dynamic events occur, making it challenging to respond promptly in real-time environments such as dynamic flexible job-shop scheduling problem (DFJSP) and dynamic job-shop scheduling problem (DJSP). Completely reactive scheduling makes on-the-fly decisions using real-time information, without predefining a complete schedule. This method typically uses priority dispatching rules (PDRs), but efficient PDRs depend heavily on human expertise and require extensive professional knowledge [8]. Additionally, the performance of PDRs is determined by the production environment and the objective function, and no single dispatching rule can achieve optimal results under all conditions [9].

Recent advancements in reinforcement learning (RL) and deep reinforcement learning (DRL) have demonstrated performance that surpasses traditional approaches, and even solvers, in solving traveling salesman problems [10] and vehicle routing problems [11]. This provides a new approach for addressing scheduling problems with dynamic events. DRL models learn through interaction with the environment, without the need for pre-collected data, and can estimate the value of future states, making them effective in handling various dynamic events in actual production workshops. Some scholars have begun exploring the use of DRL to solve dynamic scheduling problems in pursuit of more optimal solutions. Yue et al. [12] proposed a two-stage double deep Q-networks (DDQN) algorithm to address dynamic disturbances such as new job arrivals and random machine failures, solving multi-objective DFJSP through two-stage decision-making. Despite the growing adoption of DRL techniques for addressing job-shop scheduling challenges, existing studies primarily concentrate on DJSP, DFJSP, and DFJSP with additional constraints, such as DFJSP with insufficient transportation resources (DFJSP-ITR) [13] and DFJSP considering variable processing times (DFJSP-VPT) [14]. To address the aforementioned issues, an end-to-end DRL method is proposed for solving DRC-DFJSP, considering worker fatigue and proficiency. This method is capable of handling DRC-DFJSP of varying scales, filling the research gap in the application of DRL to DRC job-shop scheduling problems. The main contributions and innovations of this paper are outlined below:

(1) A DRC-DFJSP model incorporating dynamic events such as random job arrivals is constructed, taking into account heterogeneous factors like fatigue accumulation rates and multi-skilled worker proficiency. To model the system state, an innovative disjunctive graph structure is introduced.
(2) A Markov decision process (MDP) model is developed for DRC-DFJSP. Two objective functions are combined to define the reward function and state information for the agent, enabling the learning of appropriate scheduling policies.
(3) A novel disjunctive graph is proposed to capture the status of DRC-DFJSP, and an attention-based heterogeneous graph neural network is developed to efficiently transform its structure and raw information into state features for DRL agent decision-making.
(4) The proximal policy optimization (PPO) algorithm is improved by adaptive clipping and prioritized replay to accelerate convergence and enhance optimization efficiency.

The structure of this paper is organized as follows: Section 2 reviews related work. Section 3 formulates the DRC-DFJSP, incorporating worker fatigue recovery and multi-skill level factors, along with a detailed explanation of the corresponding disjunctive graph model. Section 4 elaborates on the proposed end-to-end DRL-based solution framework. Section 5 reports the experimental findings and analysis, while Section 6 summarizes the conclusions and outlines future research prospects.

2. Literature review

This section briefly reviews the human factors involved in DRC-FJSP and the conventional solution methods used to address it. Given the limited literature that considers dynamic events in DRC-FJSP, recent deep reinforcement learning (DRL) approaches applied to DFJSP are reviewed.

2.1. Human-related factors in DRC-FJSP

Recent research on DRC-FJSP focus on modeling worker flexibility, often neglecting detailed variations among individuals, including differences in skills, fatigue levels, learning capacities, and other personal attributes. These factors significantly affect actual operational efficiency, rendering workers less stable and predictable than machines. Zhang et al. [15] developed an AI-based optimization model for the DRC-FJSP that incorporates worker-specific loading and unloading time variations and employed an improved quantum genetic algorithm to enhance solution efficiency and convergence performance. Manafi et al. [16] investigated the DRC-FJSP with reconfigurable machine tools and worker resource constraints and proposed a self-learning whale optimization algorithm integrated with RL. Although studies on individual worker differences in DRC-FJSP remain limited, growing attention has been paid to worker heterogeneity under the paradigm of human-centric manufacturing, as such variability offers opportunities for improving scheduling adaptability and system resilience [17]. Han et al. [18] incorporated learning-forgetting effects and inter-worker collaboration

into a bi-objective DRC-FJSP model, optimizing both makespan and maximum workload via a hybrid metaheuristic based on multi-key-element cooperative search.

Fatigue-related studies further emphasize human heterogeneity. Research has shown that workers experience different rates of fatigue under identical workloads, which in turn degrades processing efficiency. Calzavara et al. [19] conducted a comprehensive review on labor aging in manufacturing and highlighted the insufficient exploration of aging effects on system performance and scheduling. To address the influence of worker fatigue on productivity in distributed hybrid flow shops, Song et al. [20] introduced a multi-objective evolutionary approach augmented by Q-learning, integrating a decoding mechanism to adaptively reassign tasks in response to accumulated fatigue. In practice, machines often outnumber workers as a result of rising automation levels and efforts to control labor expenses. To maintain production capacity, cross-training is commonly implemented, resulting in a workforce of multi-skilled workers who vary in skill coverage and proficiency [21]. Liu et al. [22] addressed the worker allocation issue in hybrid assembly lines by simultaneously considering multi-skill attributes and parallel team structures and developed a dual-objective optimization framework aimed at reducing both the makespan and team workload imbalance in complex assembly environments.

In summary, most existing DRC models simplify workers as fixed-efficiency resources whose contributions depend solely on predefined skill levels. However, overlooking essential heterogeneity among workers may lead to notable inconsistencies between scheduling solutions and actual production outcomes. To bridge this gap, this study proposes a human-machine coupling framework that integrates multi-skilled workers with varying proficiency and machines with different operational intensities into unified production units. This framework facilitates a more realistic evaluation of joint resource performance and enhances the practical relevance of scheduling models in human-centric smart manufacturing environments.

2.2. Methods for DRC-FJSP

Most existing research primarily focuses on solving the DRC-FJSP under static environments. The DRC-FJSP exhibits multiple complexities, and conventional solution strategies are generally categorized into three classes: analytical, simulation, and heuristic methods [15]. Analytical and simulation methods can generally provide exact solutions for medium-sized instances within a reasonable timeframe. However, the high computational complexity means that the time needed to find solutions grows exponentially as the problem size increases. In contrast, heuristic methods such as priority dispatching rules (PDR) offer advantages in terms of intuitiveness, speed, and efficiency. However, their performance is highly dependent on rule design and often lacks global optimization capability due to their myopic nature.

A recent review of scheduling problems in DRC production systems highlights a growing interest in metaheuristic methods, with genetic algorithms (GA), simulated annealing (SA), and non-dominated sorting genetic algorithm II (NSGA-II) among the most commonly employed techniques [23]. Defersha et al. [24] developed a mathematical model for DRC-FJSP based on worker resource constraints and solved it using an extended SA algorithm. Wu et al. [25] considered fixture loading/unloading times in the DRC-FJSP and used an improved NSGA-II to simultaneously minimize the makespan and fixture setup time. Meng et al. [26] established a mixed-integer programming model incorporating idle time and idle energy criteria, and introduced a variable neighborhood search approach featuring eight neighborhood structures to address the DRC-FJSP considering the different processing capabilities of workers. Li & Wu [27] focused on three sustainable development indicators, namely economic, environmental, and social, and proposed an enhanced NSGA-III algorithm guided by survival duration to address the DRC-FJSP. For the fatigue-conscious DRC-FJSP, Tan et al. [28] developed an enhanced NSGA-II algorithm, incorporating four scheduling heuristics designed to generate high-quality initial solutions, and introduced two distinct neighborhood mechanisms based on a newly defined critical path strategy to enhance local search performance.

In addition, several studies have explored the integration of RL with heuristic algorithms to address variants of the DRC problem [20], although the application of deep learning (DL) approaches to DRC-FJSP remains rare in the existing literature. Despite the success of metaheuristic algorithms in static DRC-FJSP, research on dynamic environments remains scarce. Moreover, these methods often require extensive empirical tuning of parameters and involve high computational costs due to numerous iterations, making them less effective in large-scale or real-time scenarios.

2.3. DRL methods for DFJSP

Due to frequent uncertainties in production, such as job delays and urgent insertions, DFJSP has gained attention recently. Many studies now apply DRL to solve this problem. Existing DRL approaches are typically grouped into two categories: (1) heuristic DRL-based methods and (2) end-to-end DRL methods.

DRL-based heuristic methods combine DRL with heuristic scheduling rules, offering advantages such as speed and effectiveness, making them popular in dynamic scheduling methods. Chang et al. [29] introduced a DDQN framework to solve the DFJSP with random job arrivals, demonstrating superior solution quality and generalization compared to other DL approaches. In the context of DJSP with stochastic job arrivals, Z. Wang and W. Liao [30] used 11 classic scheduling rules as actions for PPO. In general, agents typically select a scheduling rule, so the design of PDRs still significantly influences the scheduling process. However, this approach relies heavily on manually designed rules, and its effectiveness is limited by the quality of those rules. Moreover, the trained models are difficult to directly transfer to different problems, resulting in poor generalization performance.

End-to-end DRL methods have recently emerged as a promising approach. They typically leverage graph neural networks (GNNs) and their variants to extract features from graph representations (e.g., disjunctive graphs) for scheduling, and use DRL to learn effective strategies. This approach exhibits strong generalization capabilities. Lei et al. [31] introduced an RL framework using a disjunctive graph structure to optimize the makespan for large-scale DFJSPs with unpredictable job arrivals. Zhang et al. [32] introduced an end-to-end DRL architecture via a heterogeneous GNN to solve DFJSP-ITR by minimizing the makespan. However, traditional value-based algorithms such as DQN often fall short in directly optimizing scheduling strategies. Consequently, policy-based methods, especially PPO, have become increasingly prevalent in scheduling research.

To address DJSP under random processing times and machine breakdowns, Liu et al. [33] proposed a mixed graph transformer network to capture state features of dynamic disturbances and adopted alternating training of actor and critic networks in PPO to improve sample efficiency. Zhao et al. [34] developed a dual dynamic attention network to construct global representations of machines and operations, enabling end-to-end scheduling via PPO. Huang et al. [35] introduced a lightweight attention module named graph gated channel transformation to extract relevant job features during decision-making. Ding et al. [36] proposed a decentralized FJSP method based on graph convolutional networks with a multi-agent RL framework. Wan et al. [37] combined a diffusion style transformer with DRL to solve DFJSP.

Most of these studies rely on the original PPO without modification. However, PPO still has potential for improvement, especially in

Table 1
List of notations.

Notation	Description
i	Jobs index.
j	Operations index.
k	Machines index.
r	Worker index.
J	Set of jobs, $J = \{J_1, J_2, \ldots, J_n\}$.
M	Set of machines, $M = \{M_1, M_2, \ldots, M_m\}$.
W	Set of workers, $W = \{W_1, W_2, \ldots, W_l\}$.
O_i	Set of operations for job i, $O_i = \{O_{i1}, \ldots, O_{ij}, \ldots, O_{in_i}\}$.
O_{ij}	The jth operation of job i.
M_{ij}	Set of candidate machines for operation O_{ij}.
W_{ij}	Set of candidate workers for operation O_{ij}.
$FL_r(t)$	The fatigue level of worker r at time t.
λ^{kr}	The fatigue accumulation rate of worker r working on machine k.
μ^r	The fatigue recovery rate of worker r.
me_k	The machine parameter of machine k.
we_r	The worker parameter of worker r.
β_f, γ_f	The fatigue recovery constant.
δ^k	The degree parameter of influence of fatigue level on processing time on the machine k.
p_{ijk}	The preset processing time of the operation O_{ij} on machine k.
p_{ijkr}	The actual processing time of the operation O_{ij} after considering worker r.
PL_{ijr}	The proficiency level parameter of the worker r for operation O_{ij}.
AT_i	The arrival time of job i.
C_{ij}	The finishing time of O_{ij}.
MC_k	Per-unit cost on machine k.
WC_{ijr}	The unit processing cost of worker r processing operation O_{ij}.
x_{ijkr}	Binary variable equal to 1 if operationO_{ij} is assigned to worker r on machine k, 0 otherwise.

experience replay. With the adaptive learning of GNNs, PPO can flexibly modify its exploration policy to better handle complex environments. To date, no end-to-end DRL method has been developed specifically for DRC-DFJSP. Therefore, it is necessary to design dedicated graph structures and feature extraction models to support high-quality solutions.

3. Description and novel disjunction graph of the DRC-DFJSP

Worker fatigue and proficiency are considered as factors affecting operation processing times. In the context of machine heterogeneity, worker heterogeneity, and dynamic disturbances (e.g., the random arrival of new jobs), it aims to allocate appropriate processing machines and operators for each operation while minimizing the makespan and total cost as optimization objectives. This section explains the DRC-DFJSP model and its formulation, and introduces a new disjunctive graph to represent it.

3.1. Problem description

The DRC-DFJSP focuses on allocating individual jobs to specific workers and machines in the presence of dynamic events. These dynamic events can be categorized into resource dynamic events (e.g., machine failures, worker absences) and job dynamic events (e.g., random arrival of new jobs). In practical production settings, the probability of dynamic scheduling due to issues like machine failures and worker absences is relatively low. Therefore, this paper primarily focuses on job dynamic arrivals in a DRC flexible job-shop environment. The symbols for problem modeling are summarized in Table 1.

The DRC-DFJSP involves assigning n jobs J processed by l workers W on m machines M. Each job $J_i \in J$ contains n_i operations that follow certain priority relationships, forming an operation set O_i. The operation O_{ij} denotes the j-th operation of job J_i, which must be performed on a machine M_k from its compatible machine set $M_{ij} \subseteq M$. However, due to the heterogeneity of workers' skills and proficiency, the feasible machine set differs across workers. Thus, workers are directly associated with operations based on their skills, defined $W_{ij} \subseteq W$ as the set of compatible workers capable of processing operation O_{ij}, and selecting worker W_r from it to work on M_k. Different workers have varying levels of proficiency in processing O_{ij}, defined as PL_{ijr} for worker W_r. As production progresses, workers experience fatigue accumulation and recovery. W_r's fatigue level at time t defined as $FL_r(t)$. The preset processing time p_{ijk} of operation O_{ij} on machine M_k is derived from empirical data. However, the actual processing time p_{ijkr} for O_{ij} is influenced by the fatigue level $FL_r(t)$ and proficiency level PL_{ijr} of the assigned worker W_r at time t. The conceptual diagram of the DRC-FJSP is shown in Fig. 1.

The scheduling optimization objective of this paper is to choose suitable machines and workers for all operations and to optimize the job sequence, thereby minimizing the makespan C_{max} and total cost L_{cost}. These objectives are defined in Eqs. (1) and (2), respectively. From the analysis above, the assumptions that the DRC-DFJSP model must satisfy are listed in Table 2.

$$C_{max} = max_{i,j}\{C_{ij}\} \tag{1}$$

Table 2
Assumptions for DRC-DFJSP.

No.	Assumption
1	At time 0, all machines, workers and jobs are available and workers' fatigue levels are 0.
2	Each machine and worker is allowed to handle no more than one operation simultaneously.
3	The processing time preset for each operation on different machines is known.
4	Operations within the same job follow a priority order, and only one can be processed at any time.
5	The process must be executed without interruption.
6	Disregarding machine failure and non-machine processing operations.
7	The processing time range should account for auxiliary durations, such as job transfer time.
8	The processing time range should account for auxiliary durations, such as job transfer time.

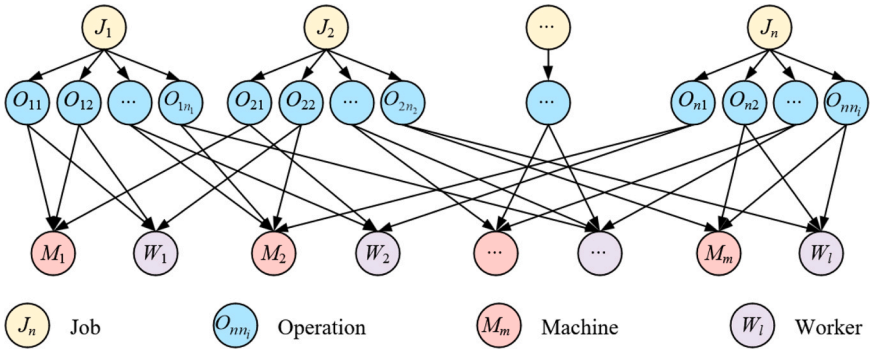
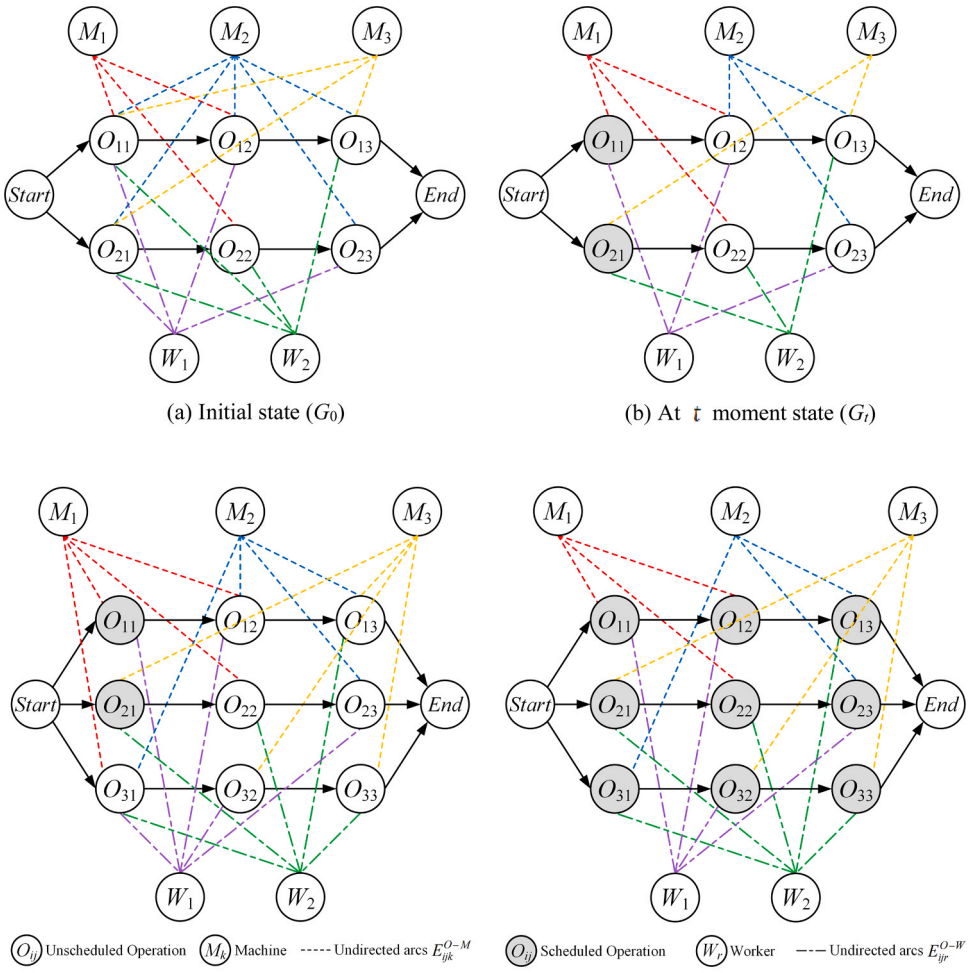


Fig. 1. Conceptual overview of DRC-FJSP.



(a) Initial state (G_0) (b) At t moment state (G_t)

(c) New job arrival (G_{t+1}) (d) Solution (G_n)

Fig. 2. Novel disjunction graph of a 3 × 3 × 2 DRC-DFJSP instance.

$$L_{cost} = \sum_{i=1}^{n} \sum_{j=1}^{n_i} \sum_{k=1}^{m} \sum_{r=1}^{l} \left(MC_k + WC_{ijr}\right) \times x_{ijkr} \times p_{ijkr} \tag{2}$$

3.2. Worker fatigue and proficiency levels

Multi-skilled workers can reduce production costs, and in the event of absenteeism, other workers can complete the tasks, ensuring production stability. However, in actual manufacturing systems, due to individual differences in worker fatigue and proficiency, different workers may take varying amounts of time and incur different labor costs when processing the same task on the same machine, ultimately affecting the overall efficiency of the production process. To enhance scheduling performance, it is essential to consider both worker fatigue and proficiency factors.

3.2.1. Worker fatigue level

Prolonged continuous labor leads to a continuous increase in workers' fatigue levels. If workers remain in a state of high fatigue for an extended period, they are highly susceptible to mental fatigue and muscle fatigue-related illnesses. In ergonomics research, some studies suggest that fatigue accumulation/recovery follows an exponential increase/decrease over time [38], while others assume it follows a linear growth/decline over time [39]. In production workshops, the operability of machines is the primary factor contributing to fatigue accumulation [40]. The rate of fatigue recovery depends on the workshop environment (such as noise, temperature, and lighting) and physiological factors (such as gender, age, and physical strength). The primary focus is on investigating the fatigue generated by heterogeneous workers operating diverse machines in flexible job-shop settings, along with fatigue's effect on production efficiency. According to M.Y. Jaber et al. [41] analysis, the exponential fatigue model demonstrates greater practicality and has found extensive application in scheduling contexts [42]. Therefore, we further refine the mathematical models for fatigue accumulation and recovery by combining machine operation characteristics [43] and workers' physical conditions [39]. The expressions for workers' fatigue during work or rest is given by Eq. (3):

$$FL_r(t_i) = \begin{cases} FL_r(t_{i-1}) + (1 - FL_r(t_{i-1}))\left(1 - exp\left(-\lambda^{kr}(t_i - t_{i-1})\right)\right), working \\ FL_r(t_{i-1}) exp\left(-\mu^r(t_i - t_{i-1})\right), resting \end{cases} \tag{3}$$

where λ^{kr} denotes the fatigue accumulation rate of W_r when operating on M_k, and μ^r represents the corresponding recovery rate. In this paper, the relevant characteristics of different machines and workers are heterogeneous, and λ^{kr} and μ^r are calculated according to Eqs. (4) and (5) respectively:

$$\lambda^{kr} = me_k \times we_r \tag{4}$$

$$\mu^r = (\beta_f - we_r)\gamma_f \tag{5}$$

where me_k is the machine M_k coefficient, and me_k is positively

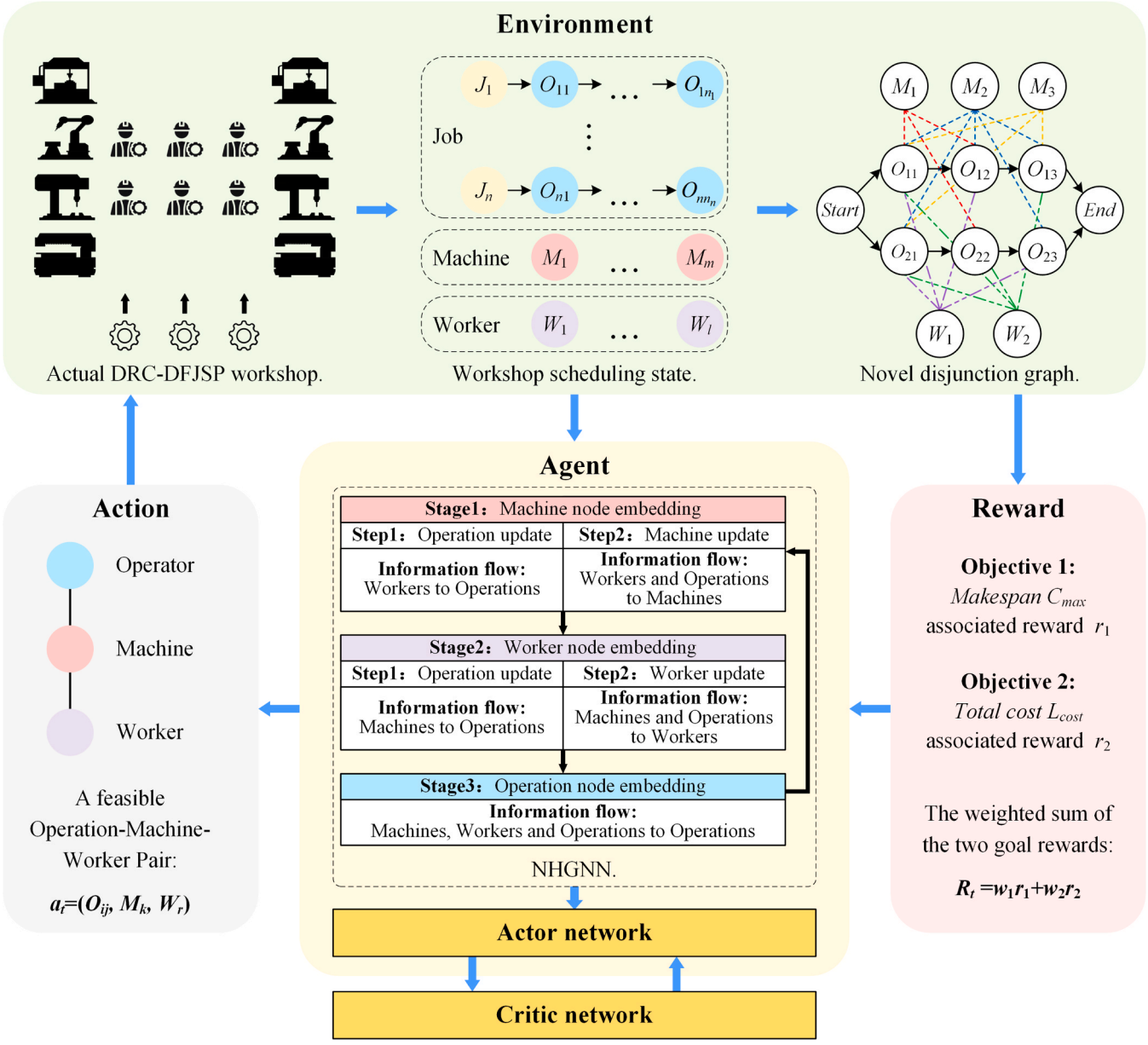


Fig. 3. Framework of proposed method.

correlated with M_k operation difficulty. we_r is the worker W_r coefficient, and we_r is negatively correlated with W_r physical condition. β_f and γ_f are constants.

3.2.2. Worker proficiency level

Multi-skilled workers, who can perform various tasks, are more flexible than single-skilled workers and enhance production system flexibility. In this paper, the proficiency of workers is divided into primary I, intermediate II, and senior III according to the process. The same worker has the skills to process a variety of operations, but there are differences in the proficiency of processing different operations. The unit processing cost and the proficiency of worker W_r processing operation O_{ij} are defined as parameters $WC_{ijr} = \{[a_I, b_I], [a_{II}, b_{II}], [a_{III}, b_{III}]\}$ and $PL_{ijr} = \{p_I, p_{II}, p_{III}\}$, respectively. The larger the PL_{ijr}, the higher the proficiency, and the higher the cost WC_{ijr}. For example, when W_r proficiency is senior III, proficiency $PL_{ijr} = p_{III}$, and cost WC_{ijr} is located in the interval $[a_{III}, b_{III}]$.

Based on the aforementioned fatigue model and proficiency, the actual processing time p_{ijk} is derived by adjusting the predefined time p_{ijkr} under the influence of worker fatigue and proficiency. The calculation process is as follows:

$$p_{ijkr} = \frac{p_{ijk}(1 + \delta^k(\ln(1 + FL_r(t))))}{PL_{ijr}} \tag{6}$$

where the parameter δ^k represents the degree to which fatigue affects processing time, and it decreases as the level of automation of machine M_k increases. When M_k is fully automated $\delta^k = 0$, indicating that processing time is not affected by worker fatigue.

3.3. Novel disjunction graph of DRC-DFJSP

The disjunctive graph intuitively captures priority constraints and machine compatibility for each operation, and has been widely used in JSP [44] and FJSP [45]. However, the DRC-DFJSP involves both selecting machines for processing operations and assigning workers to those machines. To more effectively model the state of DRC-DFJSP, a novel

disjunctive graph denoted as $G = (O, M, W, C, E)$ is proposed. This graph builds upon the work of Song et al. [46], which includes additional machine nodes and worker nodes, and expanded edge types. $O = \{O_{ij} | \forall i, j\} \cup \{Start, End\}$ contains all operation nodes, including two virtual nodes *Start* and *End*. $M = \{M_k | \forall k\}$ represents machine nodes. $W = \{W_r | \forall r\}$ is the set of worker nodes. $C = \{O_{ij}, O_{ij+1} | 1 \le i \le n, 1 \le j \le n_i - 1\}$ is the set of conjunction arcs that reflect the sequential constraint relationships between operations of the same job. Disjunctive arcs are replaced by arcs $O-M$, and additional arcs $O-W$ form the set E. Where, $E_{ijk}^{O-M} \in E$ represents the undirected arcs $O_{ij}-M_k$ between operations O_{ij} and available machines M_k, while $E_{ijr}^{O-W} \in E$ represents the undirected arcs $O_{ij}-W_r$ between operations O_{ij} and operable workers W_r.

In the dynamic scheduling process, each state s_t denoted as $G_t = (O_t, M, W, C, E_t)$, where O_t and E_t change dynamically. As illustrated in Fig. 2, a novel disjunctive graph is constructed for a $3 \times 3 \times 2$ DRC-DFJSP instance, with Fig. 2(a) showing the initial graph G_0 at time 0. After operation O_{ij} is processed by worker W_r on machine M_k, only arc E_{ijk}^{O-M} and arc E_{ijr}^{O-W} are retained, while other arcs are removed. Fig. 2(b) shows the novel disjunctive graph G_t after operations O_{11} and O_{21} have been scheduled. At this point, if a new job $J_{i'}^{new}$ arrives, all operations $O_{i'j'}$ of the $J_{i'}^{new}$ are added to G_t. Each $O_{i'j'}$ is connected to all machines $M_{k'}$ in its candidate machine set $M_{i'j'}$ with arc $E_{i'j'k'}^{O-M}$, and connected to all workers $W_{r'}$ in its candidate worker set $W_{i'j'}$ with arc $E_{i'j'r'}^{O-W}$, generating a novel disjunctive graph G_{t+1} as shown in Fig. 2(d).

The above heterogeneous graph representation offers several advantages compared to the traditional disjunctive graph used in FJSP. First, it significantly reduces the graph's density. In classic disjunctive graph representations, the number of disjunctive arcs between operation nodes increases quadratically with the number of operations, easily leading to graph density explosion and increased computational overhead. In contrast, the heterogeneous graph sparsely represents relationships between nodes through $O-M$ edges and $O-W$ edges. Specifically, each operation node only needs to connect with its candidate machine nodes and available worker nodes, leading to a linear increase in the growth of edges and a complexity of $O(n \times (m + l))$. This sparsity greatly improves computational efficiency in large-scale scheduling problems. Compared with traditional heuristic optimization algorithms that typically evaluate the entire solution set and reconstruct the scheduling graph in every generation, the proposed method, in the worst-case scenario for each scheduling decision, has a time complexity of $O(n \times m \times l)$. In terms of space complexity, the heterogeneous graph only needs to store $O(n+m+l)$ nodes and $O(n \times (m+l))$ edges, significantly reducing memory consumption compared to the $O(n^2)$ edges in disjunctive graphs.

4. Proposed method

The proposed method based on DRL is formally introduced, and the overall framework is shown in Fig. 3. First, the DRC-DFJSP is conceptualized as a DRL task, and its MDP is constructed. The agent's interaction environment is built, using the novel disjunctive graph to describe the actual scheduling state of DRC-DFJSP. Next, a three-stage novel heterogeneous graph neural network (NHGNN) is introduced to capture and encode the complex scheduling states of DRC-DFJSP. The embedded feature vectors are used to construct the actor and critic networks separately through multiple multilayer perceptrons (MLPs). Finally, the proposed NHGNN model and policy network are trained with an improved PPO algorithm.

4.1. MDP formulation of DRC-DFJSP

RL serves as a highly effective framework for addressing sequential decision-making tasks, which are typically formulated as a MDP, denoted by $M = (S, A, P, \gamma, R, \pi)$. At timestep t, the agent perceives the current state $s_t \in S$, chooses an action $a_t \in A$ based on policy π, transitions to a new state s_{t+1}, and receives an immediate reward $r_t \in R$. The objective of agent is to learn a policy π that maximizes the expected cumulative future rewards. The DRC-DFJSP problem is also expressed as an MDP characterized by the tuple $(S, A, P, \gamma, R, \pi)$, with the corresponding definitions listed as follows:

State: S is the set of states. The proposed novel disjunctive graph G_t represent the state $s_t \in S$ of the DRC-DFJSP at timestep t. s_t contains the feature information of all operations, machines, and workers at timestep t. When O_{ij} denotes the initial operation of job J_i, its start time $S_{ij}(t) = AT_i$, where AT_i is the arrival time of job J_i. Specifically, when J_i is not a newly inserted job, $AT_i = 0$. Let $\bar{p}_{ij-1} = \frac{1}{|M_{ij-1}| \times |W_{ij-1}|} \sum_{M_k \in M_{ij-1}} \sum_{W_r \in W_{ij-1}} p_{ij-1kr}$ refers to the estimated processing time of an unscheduled operation O_{ij-1}. When O_{ij} is not the first operation of J_i, and the previous operation O_{ij-1} has been processed by worker W_r on machine M_k, then $S_{ij}(t) = S_{ij-1}(t) + p_{ij-1kr}$; otherwise, $S_{ij}(t) = S_{ij-1}(t) + \bar{p}_{ij-1}$.

Action: A denotes the set of possible decisions. In the context of DRC-DFJSP, agents must simultaneously determine the operations to be executed, allocate appropriate machines, and assign workers. That is, an action $a_t \in A$ is defined as a feasible action (O_{ij}, M_k, W_r) at timestep t. Here, O_{ij} are the unscheduled operations that meet the conditions, and $M_k \in M_{ij}$ and $W_r \in W_{ij}$ are idle. If a new job $J_{i'}^{new}$ arrives before timestep $t+1$, all its operations $O_{i'j'}$, together with the idle candidate machines $M_{k'} \in M_{i'j'}$ and workers $W_{r'} \in W_{i'j'}$, form the feasible actions $a_{(t+1)} = (O_{i'j'}, M_{k'}, W_{r'})$ at timestep $t+1$, which are then added to A.

Transition: $P(s_{t+1} | s_t, a_t)$ refers to the set of state transition probabilities. At every timestep t, the agent chooses an action a_t according to the current state s_t, and interacts with the environment. If no new job arrives before timestep $t+1$, the graph G_t is updated based on a_t to generate G_{t+1}. Otherwise, if a new job $J_{i'}^{new}$ arrives, the graph G_t is updated based on the action a_t, incorporating all operations $O_{i'j'}$ of the $J_{i'}^{new}$ and information on candidate machines and workers to generate G_{t+1}. Finally, G_{t+1} is used to determine the next state s_{t+1}.

Reward: The reward function R is closely related to the objective function. To guide policy learning, two immediate rewards are defined for the optimization objectives: makespan C_{max} and total cost L_{cost}, which correspond to r_1 and r_2, respectively. As follows:

$$r_1 = C_{\max}(s_t) - C_{\max}(s_{t+1}) \tag{7}$$

$$r_2 = ratio(t) - ratio(t+1) \tag{8}$$

where $ratio(t) = \frac{TP(t)}{L_{cost}(t)}$, and $TP(t) = \sum_{i=1}^{n} \sum_{j=1}^{n_i} \sum_{k=1}^{m} \sum_{r=1}^{l} x_{ijkr} \times p_{ijkr} \times 2$, denoting the total processing time of all machines and workers. To balance the two objectives, a weight vector $R_t = \omega_1 r_1 + \omega_2 r_2$ is constructed with $\omega_1 + \omega_2 = 1$. Inspired by decomposition-based multi-objective optimization [47], iterative weight tuning based on Euclidean distance is proposed, as follow:

Step 1: Define a set of uniformly distributed weight vectors. A series of weight vectors $W_i = (\omega_1, \omega_2) \in W_{\text{weight}}$ is sampled from a predefined range between 0 and 1, including (0, 1), (0.1, 0.9), (0.2, 0.8), (0.3, 0.7), (0.4, 0.6), (0.5, 0.5), (0.6, 0.4), (0.7, 0.3), (0.8, 0.2), (0.9, 0.1) and (1, 0).

Step 2: Normalize all processing time features. To ensure consistency across instances, machine and operator processing times are normalized. These normalized values are used in both makespan and cost reward calculations.

Table 3
State raw feature vectors.

Category	Expression	Feature elements description
Operation node: O_{ij}	$x_{ij} \in R^{d_O}$, $d_O = 7$	Scheduling status: scheduled as 1, unscheduled as 0 Number of unscheduled operations for Job J_i Processing time: scheduled as p_{ijkr}, unscheduled as $\bar{p}_{ij}$ Number of candidate machines: $\lvert N_t(O_{ij}_M)\rvert$ Number of candidate workers: $\lvert N_t(O_{ij}_W)\rvert$ Start time: estimated or actual $S_{ij}(t)$ Completion time of Job J_i: estimated or actual $C_i(t)$
Machine node: M_k	$y_k \in R^{d_M}$, $d_M = 5$	Unit processing cost of machine: MC_k Machine coefficient: me_k Number of processable operations: $\lvert N_t(M_k_O)\rvert$ Available time: available time for processing O_{ij} Utilization rate:the proportion of cumulative processing time to total production time
Worker node: W_r	$z_r \in R^{d_W}$, $d_W = 5$	Worker coefficient: we_r Worker fatigue level: $FL_r(t)$ Number of processable operations: $\lvert N_t(W_r_O)\rvert$ Available time: available time for processing O_{ij} Utilization rate:the proportion of cumulative processing time to total production time
$O-M$ arc: E_{ijk}^{O-M}	$u_{ijk} \in R^{d_{O_M}}$, $d_{O_M} = 1$	Preset processing time of O_{ij} on M_k: p_{ijk}
$O-W$ arc: E_{ijr}^{O-W}	$v_{ijr} \in R^{d_{O_W}}$, $d_{O_W} = 2$	W_r proficiency level in processing O_{ij}: PL_{ijr} W_r unit processing cost for O_{ij}: WC_{ijr}

Step 3: For each weight vector W_i, perform the following: train a reinforcement learning agent using reward function R_t. Save the trained model and performance metrics.

Step 4: Initialize a candidate set of untrained weight vectors $W_{\text{remain}} \subset W_{\text{weight}}$.

Step 5: Select the next weight vector $W_{i+1} \in W_{\text{remain}}$ such that the Euclidean distance to the current weight is minimized: $\min_{W_{i+1} \in W_{\text{remain}}} \| W_{i+1} - W_i \|_2$.

Step 6: Repeat Steps 3–5 until all weight vectors in W are trained.

Policy: The policy $\pi_\theta(a_t|s_t)$ chooses an action a_t from the action space A according to a probability distribution conditioned on the immediate state s_t. θ is the parameter of the policy.

4.2. Graph structure embedding process

The end-to-end DRL method requires the extraction of effective information from the uerlying raw features of operations, machines, and workers to enable efficient decision-making by the agent. Therefore, it is crucial to extract and embed state information effectively within the novel disjunctive graph G_t. Heterogeneous GNN[48] leverages the diverse information from various node types, enhancing node feature representation. To better capture this information, a NHGNN designed for DRC-DFJSP to embed the scheduling states is proposed. First, a novel disjunctive graph G_t is used to represent the global state of DRC-DFJSP. The raw feature vectors of nodes and edges shown in Table 3 are incorporated into the constructed graph G_t, embedding the dynamically changing state information. Second, NHGNN completes the embedding in three stages: embedding for machine nodes, worker nodes, and operation nodes, respectively. The embedding process of an example is illustrated in Fig. 4. Finally, the global state embedding vector of the graph G_t is constructed and used to build the policy network.

4.2.1. Machine node embedding

To ensure that the embedding vectors of the machine nodes retain sufficient structural and feature information from the graph G_t, the feature information of the operation nodes (as first-order neighbors) and worker nodes (as second-order neighbors) is embedded into the machine node (the target node). Specifically, operation nodes are directly connected to machine nodes via $O-W$ edges, while worker nodes are indirectly connected through "worker-operation-machine" paths. Thus, the feature information of the worker and operation nodes is embedded in two sequential steps. As shown in Fig. 5, the process is as follows:

Step 1: Operation nodes update

Embed the features of the worker node W_r into the operation node O_{ij} and generate the embedding vector x_{ijw} for the enhanced operation node O_{ijw} that incorporates worker features. W_r is connected to O_{ij} through edge E_{ijr}^{O-W}. Therefore, concatenate the raw vectors of W_r and E_{ijr}^{O-W} into $z_r||v_{ijr} \in R^{d_W+d_{O_W}}$ and map them to d dimensions via a shared linear transformation $\mathbf{W}_M^{O_W} \in R^{d\times(d_W+d_{O_W})}$. Similarly, map the raw vector x_{ij} of O_{ij} to d dimensions through a shared linear transformation $\mathbf{W}_M^{O} \in R^{d\times d_O}$. The single-head attention coefficients of each worker node W_r to the operation node O_{ij} are given as follows:

$$e_{ijr} = LeakyReLU\left(a^T\left[\boldsymbol{W}_M^O x_{ij} \middle|\middle| \boldsymbol{W}_M^{O_W}\left[z_r \middle|\middle| v_{ijr}\right]\right]\right), \forall W_r \in N_t\left(O_{ij}_W\right) \tag{9}$$

where $a \in R^{2d}$, $N_t\left(O_{ij}_W\right)$ is the set of neighboring worker nodes W_r of the operation O_{ij}. The self-attention coefficient of the operation node O_{ij} is as follows:

$$e_{ij} = LeakyReLU\left(a^T\left[\boldsymbol{W}_M^O x_{ij} \middle|\middle| \boldsymbol{W}_M^O x_{ij}\right]\right) \tag{10}$$

Then, use the *softmax* function to normalize e_{ijr} and e_{ij}, obtaining the attention coefficients $(\alpha_{ijr}, \alpha_{ij})$. Finally, apply the *sigmoid* function σ to process the aggregated features of O_{ij} and all the associated W_r. This results in the embedding vector $x_{ijw} \in R^d$ for the enhanced operation node O_{ijw} that incorporates worker features. The process is as follows:

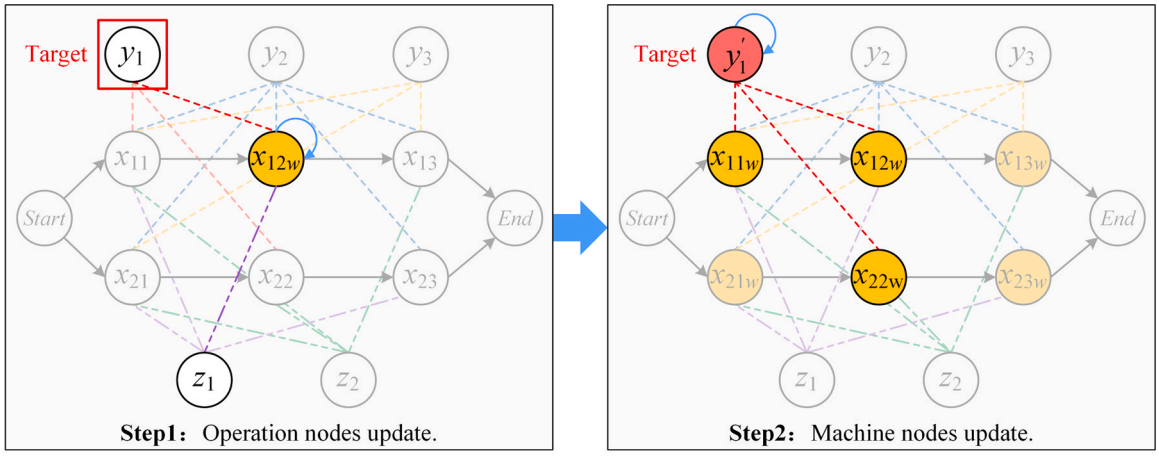
$$x_{ijw} = \sigma\left(\alpha_{ij}\boldsymbol{W}_M^O x_{ij} + \sum\nolimits_{W_r \in N_t\left(O_{ij}_W\right)} \alpha_{ijr}\boldsymbol{W}_M^{O_W}\left[z_r \middle|\middle| v_{ijr}\right]\right) \tag{11}$$

Step 2: Machine nodes update

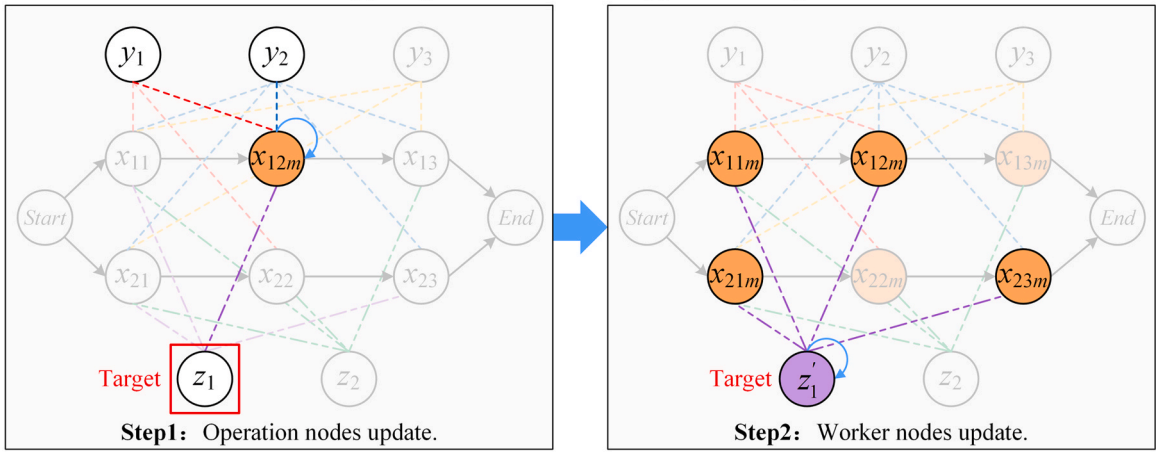
Embed the enhanced operation node O_{ijw} that incorporates worker features into the machine node M_k to obtain the embedding vector y_k' of the machine node M_k'. O_{ijw} connects to M_k via edge E_{ijk}. The single-head attention coefficients of each enhanced operation node O_{ijw} to the machine node M_k are given is computed by:

$$e_{ijk} = LeakyReLU(a^T[\boldsymbol{W}^M y_k || \boldsymbol{W}_M^{O_M}[x_{ijw}||u_{ijk}]]), \forall O_{ij} \in N_t(M_k_O) \tag{12}$$

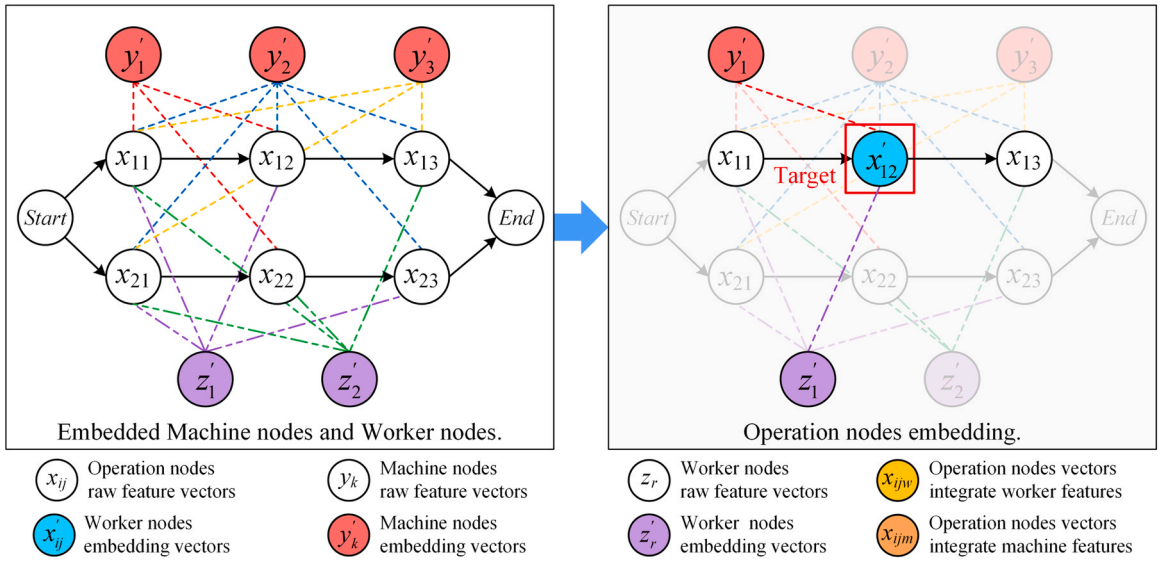
where $\mathbf{W}^M \in R^{d\times(d_M)}$ and $\mathbf{W}_M^{O_M} \in R^{d\times(d+d_{O_M})}$ are the shared linear matrices, and $N_t(M_k_O)$ denotes the operation nodes O_{ijw} neighboring the machine node M_k. The formula for calculating the self-attention score for M_k is as follows:



(a) Stage 1. Machine node y_1 embedding.



(b) Stage 2. Worker node z_1 embedding.



(c) Stage 3. Operation x_{12} node embedding.

Fig. 4. Three-stage embedding process of a 2 × 3× 2 DRC-FJSP instance.

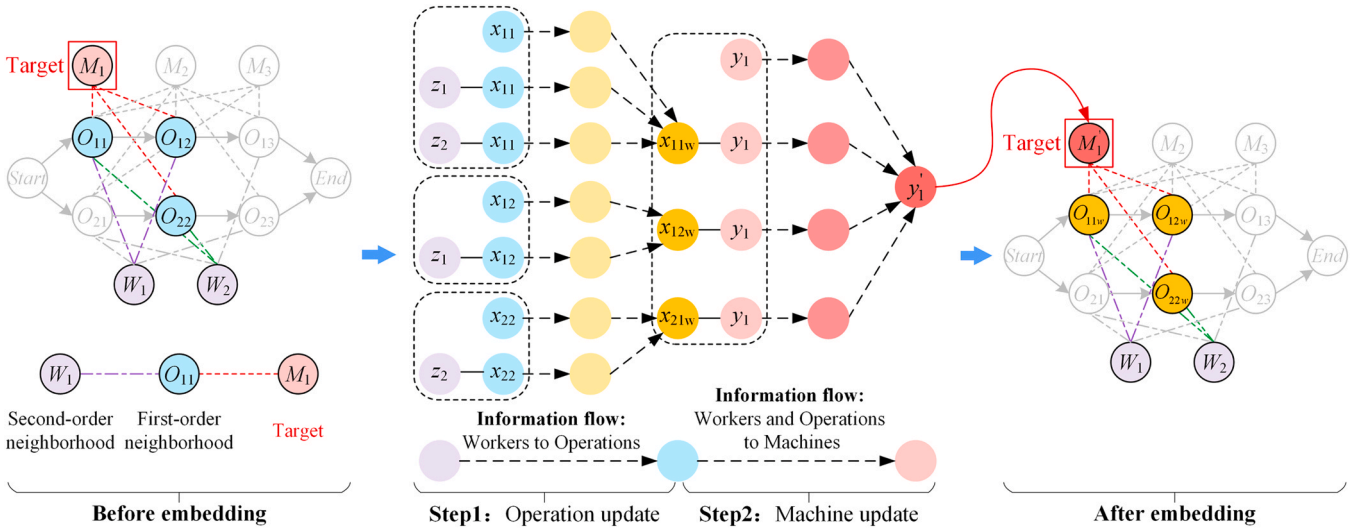


Fig. 5. Embedding process of a target machine node.

$$e_k = LeakyReLU\left(a^T\left[\boldsymbol{W}^M y_k \middle|\middle| \boldsymbol{W}^M y_k\right]\right) \tag{13}$$

Then, e_{ijk} and e_k are normalized to derive the attention weights (α_{ijk}, α_k). Finally, aggregate the features of the M_k and all the associated O_{ijw} to obtain the embedding vector $y_k' \in R^d$ of the M_k'. The process is as follows:

$$y_k' = \sigma\left(\alpha_k \boldsymbol{W}^M y_k + \sum_{O_{ij}\in N_t\left(M_k_O\right)} \alpha_{ijk} \boldsymbol{W}_M^{O_M}\left[x_{ijw} \middle|\middle| u_{ijk}\right]\right) \tag{14}$$

4.2.2. Worker node embedding

Similar to the machine node, the worker node (target node) also integrates the feature information from the second-order neighbors (machine nodes) and its first-order neighbors (operation nodes) through two steps.

Step 1: Operation nodes update

Calculate the single-head attention coefficients f_{ijk} of each machine node M_k to the operation node O_{ij}, as well as the self-attention coefficient f_{ij} of O_{ij}. Normalize to obtain $\left(\beta_{ijk}, \beta_{ij}\right)$. Aggregate the features of the O_{ij} and all the associated M_k to get the embedding vector $x_{ijm} \in R^d$ of the enhanced operation node O_{ijm} that incorporates machine features. The calculation process is as follows:

$$x_{ijm} = \sigma\left(\beta_{ij} \boldsymbol{W}_W^O x_{ij} + \sum_{M_k\in N_t\left(O_{ij}_M\right)} \beta_{ijk} \boldsymbol{W}_W^{O_M}\left[y_k \middle|\middle| u_{ijk}\right]\right) \tag{15}$$

where $\mathbf{W}_W^O \in R^{d\times d_O}$ and $\mathbf{W}_W^{O_M} \in R^{d\times\left(d_O+d_{O_M}\right)}$ are the shared linear matrices, and $N_t\left(O_{ij}_M\right)$ represents the set of machine nodes M_k neighboring the operation node O_{ij}.

Step 2: Worker nodes update

Calculate the single-head attention coefficients f_{ijr} of each enhanced operation node O_{ijm} to the worker node W_r, as well as the self-attention coefficient f_r of W_r. Normalize to obtain $\left(\beta_{ijr}, \beta_r\right)$. Aggregate the features of the W_r and all the associated O_{ijm} to get the embedding vector $z_r' \in R^d$ of the worker node W_r'. The process is as follows:

$$z_r' = \sigma\left(\beta_r \boldsymbol{W}^W z_r + \sum_{O_{ij}\in N_t\left(W_r_O\right)} \beta_{ijr} \boldsymbol{W}_W^{O_W}\left[x_{ijm} \middle|\middle| v_{ijr}\right]\right) \tag{16}$$

where $\mathbf{W}^W \in R^{d\times d_W}$ and $\mathbf{W}_W^{O_W} \in R^{d\times\left(d+d_{O_W}\right)}$ are the shared linear matrices, and $N_t(W_r_O)$ represents the set of operation nodes O_{ijm} neighboring the worker node W_r.

4.2.3. Operation node embedding

Due to the diverse arc types linked to operation nodes, attention mechanisms are less effective for embedding [46]. Therefore, based on the completed embedding of the machine nodes and worker nodes, MLP will be used for the embedding of the operation node.

Define six MLPs (denoted as $MLP_{\omega_0}, \ldots, MLP_{\omega_5}$) to process the feature vectors x_{ij-1}, x_{ij}, x_{ij+1}, y_k' and z_r', respectively. Each MLP_{ω_h} $(h = 0, \ldots, 5)$ has two d_ω-dimensional hidden layers, and uses exponential linear unit (ELU) activation. Then, concatenate the processed results and apply ELU activation followed by MLP processing. The final result is the embedding vector $x_{ij}' \in R^d$ of the operation node O_{ij}. The embedding process is as follows:

$$x_{ij}' = MLP_{\omega_0}\left(ELU\begin{bmatrix} MLP_{\omega_1}\left(x_{ij-1}\right)\middle|\middle|MLP_{\omega_2}\left(x_{ij+1}\right)\middle|\middle| \\ MLP_{\omega_3}\left(\bar{y}_k'\right)\middle|\middle|MLP_{\omega_4}\left(\bar{z}_r'\right)\middle|\middle|MLP_{\omega_5}\left(x_{ij}\right)\end{bmatrix}\right) \tag{17}$$

where x_{ij-1}, x_{ij} and x_{ij+1} are the raw feature vectors of O_{ij-1}, O_{ij} and O_{ij+1}, respectively. $\bar{y}_k' = \sum_{M_k\in N_t\left(O_{ij}_M\right)} y_k'$ is the average of the sum of the embedding vectors y_k' of all neighboring machine nodes M_k' of O_{ij}. $\bar{z}_r' = \sum_{W_r\in N_t\left(O_{ij}_W\right)} z_r'$ is the average of the sum of the embedding vectors z_r' of all neighboring worker nodes W_r' of O_{ij}. $||$ is a serial connector.

4.2.4. Graph-level embedding

The aforementioned embedding process can be regarded as an embedding layer. The NHGNN model is constructed by stacking N embedding layers with the same structure but independent and trainable parameters, obtaining the final embedding vectors $x_{ij}^{(N)}$, $y_k^{(N)}$, and $z_r^{(N)}$. The averages of the three types of node embedding vectors are calculated in accordance with the commonly used pooling method and concatenated, eventually obtaining the global state embedding vector $g_t \in R^{3d}$ of the graph G_t. The process is as follows:

$$g_t = \left[\frac{1}{|O|}\sum_{O_{ij}\in O} x_{ij}^{(N)} \middle|\middle| \frac{1}{|M|}\sum_{M_k\in M} y_k^{(N)} \middle|\middle| \frac{1}{|W|}\sum_{W_r\in W} z_r^{(N)}\right] \tag{18}$$

where $|O|$, $|M|$, and $|W|$ represent the number of operations, machines, and workers, respectively, in the DRC-DFJSP instance.

4.3. Decision making based on NHGNN

To select an action a_t in state s_t, this paper designs a policy network $\pi_\theta(a_t|s_t)$ based on the graph embedding vector g_t extracted by the

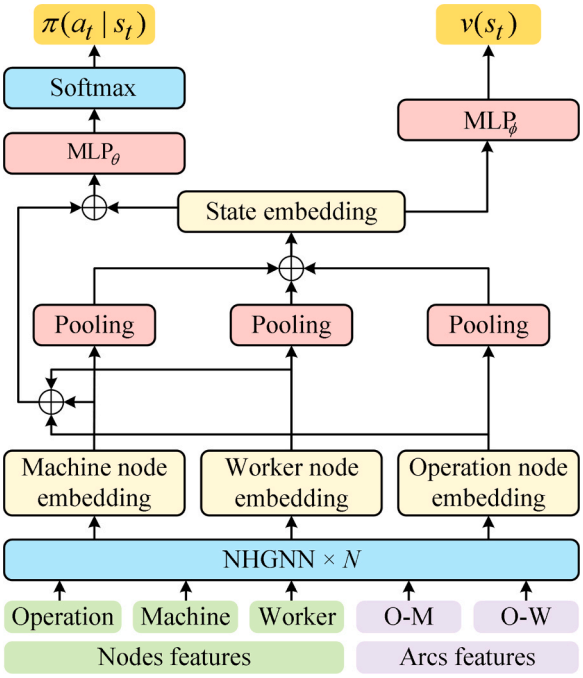


Fig. 6. Overall framework of the proposed NHGNN.

NHGNN model. First, at each timestep t, the embedding features $x_{ij}^{(N)}$, $y_k^{(N)}$, and $z_r^{(N)}$ corresponding to each feasible action $a_t = (O_{ij}, M_k, W_r) \in A_t$ are concatenated with the global state embedding vector g_t. These are then processed using an MLP_θ with two d_θ-dimensional hidden layers and *tanh* activation to obtain the probability distribution for selecting action a_t in state s_t:

$$P(a_t, s_t) = MLP_\theta\left[x_{ij}^{(N)} \| y_k^{(N)} \| z_r^{(N)} \| g_t\right] \tag{19}$$

Then, the *softmax* function is applied to normalize $P(a_t, s_t)$:

$$\pi_\theta\left(a_t | s_t\right) = \frac{exp(P(a_t, s_t))}{\sum_{a_t' \in A_t} exp\left(P\left(a_t', s_t\right)\right)} \tag{20}$$

Finally, different decoding strategies are used to predict action a_t. During training, actions are randomly sampled based on the policy π_θ helping the agent explore the environment and obtain a better-trained model. During testing, a greedy strategy is adopted, selecting the action with the highest probability in each state to seek the optimal policy.

4.4. Improved Proximal policy optimization

To enable real-time resolution of the constructed MDP model, an improved PPO algorithm is proposed. Compared with other RL algorithms, PPO [49] has the advantages of low sensitivity to the learning rate and fewer parameters to tune, and has therefore been widely adopted in dynamic and continuously changing environments. An improved variant, referred to as PER-AC-PPO (PPO with prioritized experience replay and adaptive clipping), is developed to enhance its applicability in dynamic scheduling environments. This algorithm integrates two key mechanisms: an adaptive clipping strategy and a prioritized experience replay mechanism, which are designed to improve both policy stability and sample efficiency. The adaptive clipping mechanism dynamically adjusts the clipping coefficient c_p during training to better balance exploration and exploitation. Inspired by learning rate decay strategies in DRL, the clipping coefficient decays exponentially over time from an initial value $c_{p_0} = 0.2$ to a final value $c_{p_1} = 0.05$, following the function:

$$c_p = \boldsymbol{exp}\left(-\frac{t}{T} ln\left(\frac{c_{p_0}}{c_{p_1}}\right)\right) c_{p_0} \tag{21}$$

where t is the current training step, T is the total training steps, and $\ln(\bullet)$ denotes the natural logarithm.

In addition, a prioritized experience replay mechanism is introduced to address the inefficiency of uniform sampling in PPO. Experience samples are assigned priorities based on their dynamic objective values, calculated according to the cumulative reward function, enabling the algorithm to focus on more informative experiences. The sampling probability for each sample is defined as:

$$p_w = \frac{\frac{1}{c_w}}{\sum_{w=1}^{P} \frac{1}{c_w}} \tag{22}$$

where P is the total number of experience samples, and c_w denotes the cumulative reward function associated with sample w. This prioritization increases the likelihood of selecting high-quality samples, thereby improving data efficiency and accelerating convergence. The PER-AC-PPO algorithm with an actor-critic architecture is employed, where both networks are updated after a fixed number of timesteps without requiring a complete trajectory, thus accelerating training. The critic shares the same NHGNN architecture as the actor. The actor refers to the aforementioned policy network π_θ. The critic v_ϕ uses an MLP_ϕ with two d_ϕ-dimensional hidden layers and tanh activation to process the graph embedding vector g_t and output a scalar to estimate the cumulative return at state s_t. Essentially, v_ϕ serves as an evaluator that learns the state value function approximation $v_\phi(s_t)$, enabling the calculation of the variance-reduced advantage function estimator A_t as follows:

$$A_t = \sum_{t'=t}^{T} \gamma^{t'} r_{t'} - v_\phi(s_t) \tag{23}$$

where γ is the discount factor, t is the current timestep, and $T \ll |O|$. The complete network architecture is illustrated in Fig. 6.

During each training iteration, the actor interacts with the environment to gather experience tuples, and the actor π_θ and critic v_ϕ are updated based on the collected samples. Specifically, the actor network is trained to maximize the loss function $L_{LOSS}(\theta)$ as follows:

$$L_{LOSS}(\theta) = c_p L_{CLIP}(\theta) + c_e L_E(\theta) \tag{24}$$

where $c_e = 0.01$ is entropy coefficient hyperparameter, $L_{CLIP}(\theta)$ is clipped surrogate objective and $L_E(\theta)$ is entropy objective. They are calculated respectively as follows:

$$L_{CLIP}(\theta) = E_t[min\{\delta_t(\theta)A_t, clip(\delta_t(\theta), 1-\epsilon, 1+\epsilon)A_t\}] \tag{25}$$

$$L_E(\theta) = E_t[Entropy(\pi_\theta(a_t|s_t))] \tag{26}$$

where $\delta_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio of target network π_θ to the behavior policy $\pi_{\theta_{old}}$ used for sampling and $\epsilon \in (0, 1)$ is the clipping parameter. The critic network undergoes updates by minimize the mean squared error (MSE) objective $L_{MSE}(\phi)$, calculated as follows:

$$L_{MSE}(\phi) = E_t\left[MSE\left(r_t, v_\phi(s_t)\right)\right] \tag{27}$$

where r_t is the reward at timestep t.

The pseudocode for the training process is shown in Algorithm 1. B instances are generated from a uniform distribution (line 3), and the behavior actor engages with the environment to schedule each instance (lines 5–15), collecting experience tuples until scheduling is completed for all tasks. Specifically, when the last operation of the instance is scheduled, the episode ends (line 11). At this point, the total number of operations in the instance $|O|$ corresponds to the number of episodes. Then, the probability ratio $\delta_t(\theta)$, the error $L_{LOSS}(\theta)$ of the target actor network, and the error $L_{MSE}(\phi)$ of the critic network are computed, and the networks are updated E_u times using a gradient ascent method (lines 16–21). Finally, the target actor network θ is copied to the behavior

actor network θ_{old} (line 22). This completes one iteration. The training process consists of E_t iterations, with validation performed on an independent validation set every e_{val} iterations (lines 23–25) and a batch of B instances replaced every e_{rep} iterations (lines 26–28).

Algorithm 1. PER-AC-PPO for DRC-DFJSP

1: **Input:** Training steps E_t, network update steps E_u, and batch size B; $NHGNN$ model, target actor network π_θ and critic network v_ϕ with initial parameters ω, θ and ϕ; initial behavior actor network $\pi_{\theta_{old}}$ by $\theta_{old} \leftarrow \theta$; fixed replace batch data e_{rep} and validation data e_{val}.
2: **Output:** Trained parameter sets $\{\omega, \theta, \phi\}$ of all networks.
3: Sample a batch of B DRC-DFJSP instances;
4: **for** $e_t = 1, \cdots, E_t$ **do**
5: **for** $b = 1, \cdots, B$ **do**
6: **for** $t = 0, 1, \cdots$ **do**
7: Extract embedding using $NHGNN$;
8: Sample a_t according to $\pi_{\theta_{old}}(a_t \mid s_t)$;
9: Observe reward r_t and next state s_{t+1};
10: Compute the generalized advantage estimates A;
11: **if** s_t is terminal **then**
12: break;
13: **end**
14: **end**
15: **end**
16: **for** $e_u = 1, \cdots, E_u$ **do**
17: Compute the probability ratio $\delta_t(\theta)$, $L_{CLIP}(\theta)$ and $L_E(\theta)$;
18: Compute the actor loss: $L(\theta)$ and the critic loss: $L_{MSE}(\phi)$;
19: Update the parameters ω, θ and ϕ by a gradient;
20: **end**
21: Update behavior actor network by $\theta_{old} \leftarrow \theta$;
22: **if** $e_t \bmod e_{val} = 0$ **then**
23: Validate the policy;
24: **end**
25: **if** $e_u \bmod e_{rep} = 0$ **then**
26: Sample a new batch of B DRC-DFJSP instances;
27: **end**
28: **end**

5. Computational experimental

5.1. Experimental environment

To validate the effectiveness of the proposed algorithm, both expanded benchmark instances and real-world cases were used. Since no public DCR-DFJSP datasets are available, benchmark instances were

Table 4
Details of DRC-FJSP Instance Scales.

Instance	Scale ($n \times m \times l \times q$)	Case	Scale ($n \times m \times l \times q$)
E01	$10 \times 4 \times 3 \times 10$	K01	$10 \times 12 \times 8 \times 5$
E02	$10 \times 6 \times 4 \times 10$	K02	$10 \times 12 \times 8 \times 30$
E03	$15 \times 15 \times 8 \times 20$	K03	$15 \times 12 \times 8 \times 15$
E04	$20 \times 10 \times 8 \times 30$	K04	$15 \times 12 \times 8 \times 25$
E05	$30 \times 15 \times 10 \times 30$	K05	$30 \times 12 \times 8 \times 40$
E06	$40 \times 20 \times 12 \times 30$	K06	$40 \times 12 \times 8 \times 50$

constructed based on the MK set proposed by Brandimarte [50], including six cases of varying sizes (E01-E06) as shown in Table 4. The processing time p_{ijk} was inherited from the benchmark, and the unit time cost MC_{ijk} for each machine was sampled from a uniform distribution $U_{MC}[2,5]$. The worker set was extended for each instance. Each operation was assigned at least two candidate workers, and each worker was assigned no fewer than two operations. Workers were randomly classified into three proficiency levels (I/II/III), with priority weights $PL_{ijr} = \{p_I = 0.8, p_{II} = 1.0, p_{III} = 1.2\}$, and their unit costs WC_{ijr} were sampled from uniform distributions $U_I[0.07, 1.12]$, $U_{II}[1.10, 1.20]$, $U_{III}[1.18, 1.23]$, respectively. The arrival time of new jobs was randomly generated, with the average arrival interval sampled from a uniform distribution $\beta_p = \{20,100\}$. In each dynamic event, q newly arrived jobs were inserted into the system. The fatigue coefficients for machines me_k and for workers we_r were sampled from uniform distributions $U_{me}[0.05, 0.20]$ and $U_{we}[0.5, 1.0]$, respectively. The degree parameter δ^k was sampled from the range $\{0, 0.3/\ln 2, 0.7/\ln 2\}$ [40]. Unless otherwise specified, the expansion parameters were set as $\beta_f = 1.4$, $\gamma_f = 0.05$.

In the real-world case study, a flexible workshop in an aerospace structural component manufacturing enterprise is used as an example. The workshop has 12 machines and 8 workers, capable of processing 10 types of operations. As shown in Table 5, each machine is capable of processing at least one operation type, and each worker has the skills to perform at least three types of operations. The workshop considers three levels of efficiency, corresponding to the proficiency parameters, which are consistent with the previously defined ranges and values. The unit processing costs of machines and workers, as well as the fatigue and recovery-related information, are predetermined, as shown in Table 6 and Table 7. High-precision machines are highly automated, and worker fatigue does not affect processing efficiency $\delta = 0$. Coarse processing machines require collaboration between workers and automated equipment, and worker fatigue has a certain degree of impact on processing efficiency $\delta = 0.3/ln2$. Non-automated and quality inspection

Table 5
Operations and alternative machines and workers of the case.

Num	Operations	p_{ijk}	Machines	Workers/Skill Lever
1	Rough turning−1	[7,8,10,16]	M_1, M_2, M_3, M_4	W_1/II, W_2/II, W_3/III
2	Rough turning−2	[12,16,18,19]	M_1, M_2, M_3, M_4	W_1/II, W_2/III, W_3/II
3	Fine turning	[10,13]	M_1, M_2	W_1/III, W_2/I, W_8/II
4	Utter-fine turning	[6,7]	M_1, M_2	W_1/II, W_2/I, W_8/III
5	Boring	[14]	M_5	W_4/I, W_6/III
6	Rough milling	[9,10,13]	M_6, M_7, M_8	W_4/II, M_5/III, W_6/I
7	Fine milling	[14,15]	M_7, M_8	W_4/III, M_5/II, W_7/I
8	Rough grinding	[14,15]	M_9, M_{10}	M_5/II, W_6/I, W_7/III
9	Coating	[15,17]	M_{11}, M_{12}	W_4/II, W_6/III, W_7/I
10	Flaw detection	[16,18]	M_{11}, M_{12}	W_3/I, W_7/II, W_8/III

Table 6
Machines related information.

Machine	me_k	MC_{ijk}	δ^k	Machine	me_k	MC_{ijk}	δ^k
M_1	0.05	3.2	0	M_7	0.16	2.9	0
M_2	0.08	3.0	0	M_8	0.14	3.0	0
M_3	0.18	2.5	$0.3/ln2$	M_9	0.12	2.6	$0.3/ln2$
M_4	0.10	2.8	$0.3/ln2$	M_{10}	0.18	2.7	$0.3/ln2$
M_5	0.12	2.6	$0.7/ln2$	M_{11}	0.16	2.9	$0.7/ln2$
M_6	0.18	2.7	$0.3/ln2$	M_{12}	0.14	3.0	$0.7/ln2$

equipment requires workers to operate and make judgments, significantly affecting processing efficiency due to fatigue $\delta = 0.7/ln2$ [40]. Additionally, the fatigue recovery constant is set to $\beta_f = 1.4$ and $\gamma_f = 0.05$. At a certain time, there are 10 jobs awaiting processing, and each job includes multiple operations, with at least four operations required for completion. The layout of the job shop is shown in Fig. 7. To comprehensively evaluate the performance of the proposed model, six scheduling cases (K01-K06) were designed by varying the number of initial jobs and newly arriving jobs as shown in Table 4.

5.2. Model performance index

Three multi-objective optimization evaluation metrics are used to assess the performance of the algorithm: *GD* [50], *IGD* [51], and *Spread* [52]. *GD* (defined in Eq. (28)) and *Spread* (defined in Eq. (29)) are used to evaluate the convergence and diversity of the algorithm, respectively. *IGD* (defined in Eq. (30)) evaluates the algorithm comprehensively in terms of both convergence and diversity.

$$GD(A, X^*) = \frac{\sqrt{\sum_{i=1}^{|A|} D_i^2(A, X^*)}}{|A|} \tag{28}$$

where X^* is the true Pareto front (true-PF), A is used to evaluate the optimal Pareto front obtained by the algorithm and $D_i(A, X^*)$ indicates the minimum Euclidean distance between solution i in A and the solutions in X^*.

$$Spread = \frac{\sum_{j=1}^{n_0} D_j^e(A, X^*) + \sum_{i=1}^{|A|} |D_i(A, A) - \overline{D}_i(A, A)|}{\sum_{j=1}^{N_0} D_j^e(A, X^*) + |A|\overline{D}_i(A, A)} \tag{29}$$

where $D_i(A, A)$ is the Euclidean distance from the *i-th* solution in A to the nearest solution in A, and $\overline{D}_i(A, A)$ is the average of $D_i(A, A)$. $D_j^e(A, X^*)$ is the Euclidean distance between the extreme solution of A under the j-th objective and the boundary solution of X^*. n_0 is the number of objective functions:

$$IGD(X^*, A) = \frac{\sum_{i=1}^{|X^*|} D_i(X^*, A)}{|X^*|} \tag{30}$$

where $D_i(X^*, A)$ indicates the minimum Euclidean distance between the *i-th* solution in X^* and the solutions in A. Smaller values of *GD*, *IGD*, and *Spread* indicate improved convergence and diversity of the algorithm. In the dynamic scheduling problem examined in this paper, the true Pareto front is unknown; therefore, the Pareto front created from the solutions of all comparison algorithms is considered the true Pareto front.

In practical production scenarios, the Pareto-optimal set often fails to support direct decision-making, necessitating the selection of a most satisfactory compromise solution. multi-criteria decision-making (MCDM) methods are commonly used to address this issue, as they consider multiple evaluation criteria simultaneously and enable decision-makers to express preferences through weight assignments. Typical MCDM methods include weighted sum method (WSM) [53], analytic hierarchy process (AHP) [54], ELECTRE [55], fuzzy membership functions [56], and TOPSIS [57]. WSM and AHP rely on clearly defined weight assignments, making them sensitive to subjectivity.

Table 7
Workers related information.

Worker	we_r	WC_{ijr}			Worker	we_r	WC_{ijr}		
		I	II	III			I	II	III
W_1	0.75	1.91	2.13	2.21	W_5	0.70	-	2.16	2.23
W_2	0.50	1.45	2.11	2.23	W_6	0.45	2.03	-	2.21
W_3	0.60	1.6	2.12	2.19	W_7	0.80	1.64	2.11	2.18
W_4	0.55	1.16	2.10	2.21	W_8	0.65	1.13	2.18	2.23

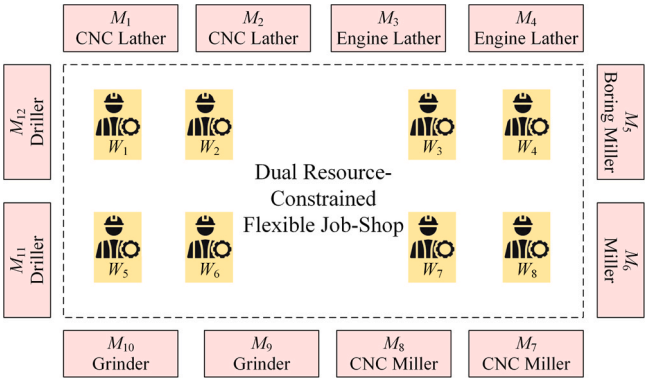


Fig. 7. Schematic diagram of the layout of the processing DRC-DFJSP.

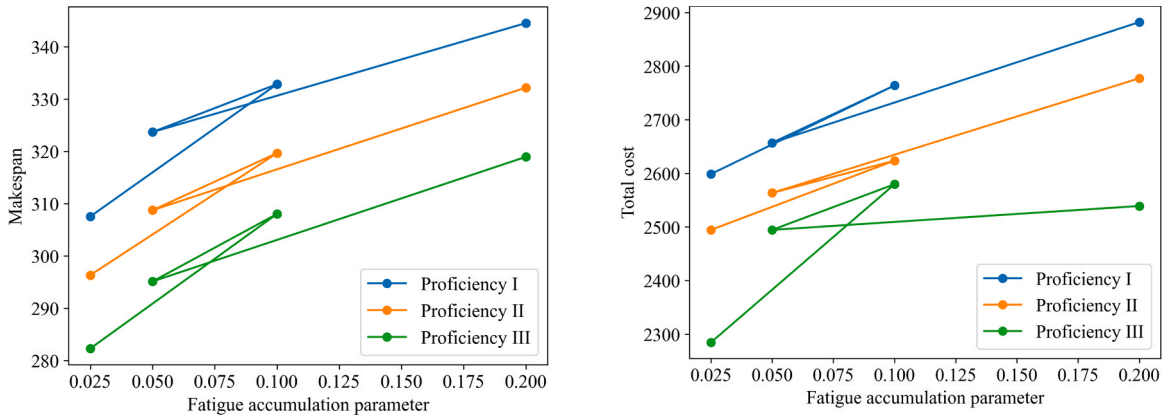
Although ELECTRE can handle uncertainty, it involves complex threshold settings. TOPSIS ranks alternatives based on their distance to the ideal solution but assumes mutual independence between objectives. This assumption is not satisfied in this study, as confirmed by Pearson correlation analysis. In contrast, the fuzzy membership function method transforms each objective value into a satisfaction degree, making it suitable for modeling nonlinear relationships and fuzzy preferences. Therefore, this study adopts the fuzzy membership function to calculate the satisfaction of each solution in the Pareto set and identifies the most satisfactory compromise solution accordingly. The formulation is presented as follows [58]:

$$\mu_i = \begin{cases} 1, f_i < f_{i\,min} \\ \dfrac{f_{i\,max} - f_i}{f_{i\,max} - f_{i\,min}}, f_{i\,min} \le f_i \le f_{i\,max} \\ 0, f_{i\,max} < f_i \end{cases} \tag{31}$$

where μ_i represents the satisfaction degree with respect to the i-th

Table 8
Sensitivity analysis of fatigue accumulation rate and and proficiency levels.

me_k	we_k	WC_{ijr}	C_{max}	L_{cost}	me_k	we_k	WC_{ijr}	C_{max}	L_{cost}
0.05	0.5	I	307.54	2598.71	0.05	1	I	323.68	2657.00
0.05	0.5	II	296.33	2494.41	0.05	1	II	308.74	2563.41
0.05	0.5	III	282.31	2284.45	0.05	1	III	295.12	2494.41
0.2	0.5	I	332.82	2763.87	0.2	1	I	344.50	2882.20
0.2	0.5	II	319.64	2623.57	0.2	1	II	332.16	2777.27
0.2	0.5	III	308.02	2579.51	0.2	1	III	318.93	2538.98



(a) Using makespan as the objective (b) Using cost as the objective

Fig. 8. Sensitivity plots of fatigue accumulation rate and and proficiency levels.

Table 9
Hyperparameter information and optimization results.

Hyperparameter	Candidate Values	Optimal Value
Learning rate (*lr*)	[0.01, 0.001, 0.0001]	0.0001
Number of MLP Layers (MLP)	[1–4]	3
Number of hidden dim (dim)	[32, 64, 128]	64

objective function, f_i denotes the value of the i-th objective, and $f_{i\,\min}$ and $f_{i\,\max}$ are the minimum and maximum values of the i-th objective function within the solution set, respectively. The overall satisfaction degree μ is calculated using Eq. (32). A higher μ value (closer to 1) indicates greater satisfaction, while a lower μ value (closer to 0) implies less satisfaction. The solution with the highest overall satisfaction degree is ultimately selected as the optimal compromise solution.

$$\mu = \frac{1}{S}\sum_{i=1}^{S}\mu_i \tag{32}$$

where S denotes the number of objective functions. In this study, S is equal to two, as the problem involves two objectives.

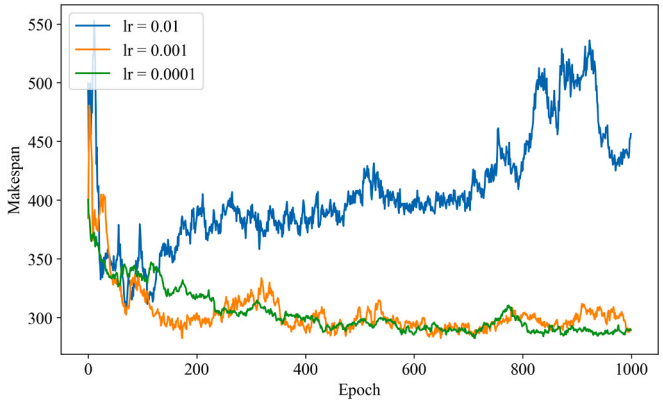
5.3. Experimental results and analysis

To ensure fairness in algorithm comparison, all experiments were conducted under the same hardware and software environments. Python 3.12 and the PyTorch 1.8.2 framework were used as the experimental environment for the proposed algorithm. The hardware configuration includes a 13th Gen Intel(R) Core(TM) i7-13700F, 32 GB RAM, and an NVIDIA GeForce RTX 4060 Ti.

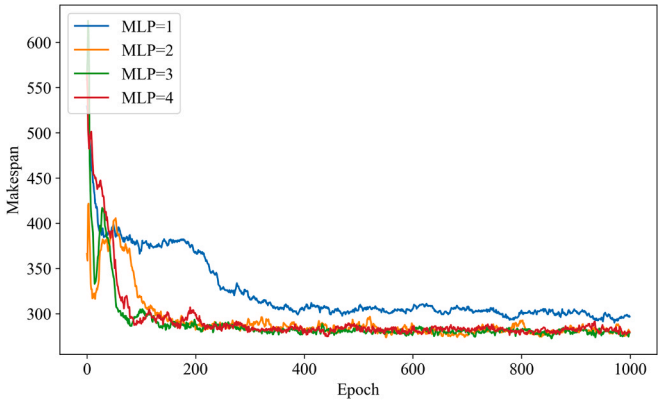
5.3.1. Sensitivity Analysis

To analyze the impact of fatigue accumulation rate and proficiency levels on makespan and cost, Case E01 is selected as a representative scenario. Multiple parameter combinations are considered, and the compromise solution derived from the fuzzy membership function serves as a reference for comparison. The fatigue accumulation rate is defined as the product of machine and operator coefficients (defined in Eq. (4)). In the experiment, machine coefficients me_k are set to 0.05 and 0.20, while operator coefficients we_r are set to 0.5 and 1.0. Corresponding proficiency levels are defined as $p_I = 0.8$, $p_{II} = 1$, $p_{III} = 1.2$, with cost coefficients WC_{ijr} sampled accordingly. As shown in Table 8, the sensitivity analysis indicates that both the proficiency level and the fatigue accumulation rate significantly affect system performance. This trend is further illustrated in Fig. 8, where the makespan and total cost are plotted against varying fatigue levels under different proficiency levels. Specifically, when the proficiency level increases from I to III, both makespan and total cost decrease noticeably, indicating that skilled workers can improve operational efficiency and reduce overall costs. In contrast, under a fixed proficiency level, increasing fatigue coefficients leads to prolonged makespan and elevated cost, demonstrating the negative impact of fatigue on system performance. These findings highlight the importance of jointly considering both proficiency and fatigue factors in the parameter tuning process to ensure efficient and stable production scheduling.

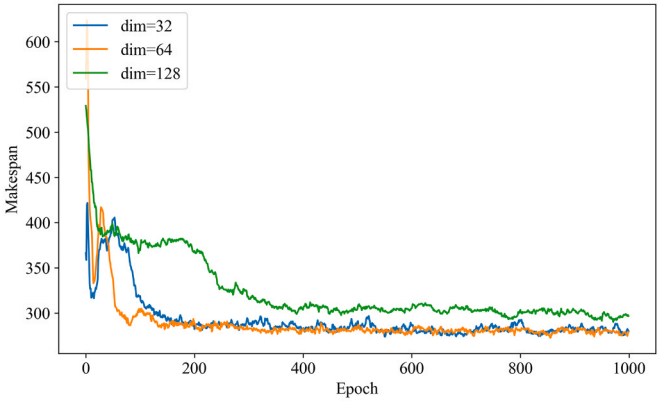
The choice of hyperparameters significantly affects both the convergence speed and overall performance. Key hyperparameters, such as learning rate, number of MLP layers, and number of hidden units, were fine-tuned using grid search. The optimal configuration, summarized in Table 9, was determined with instance E01 under a weight setting of (1, 0). The iteration curves for makespan under different configurations are shown in Fig. 9. Learning rates of 0.01, 0.001, and



(c) Learning rate

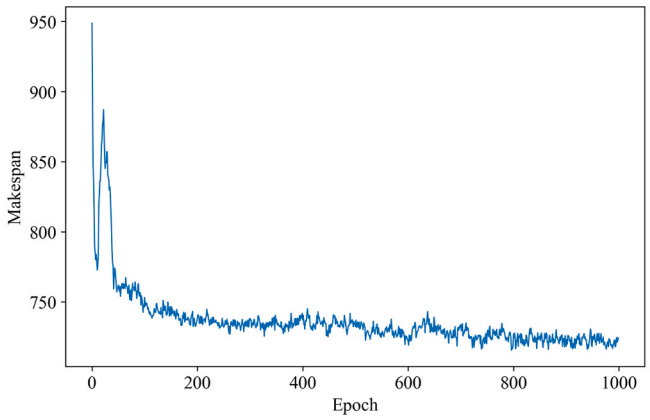


(d) Number of MLP Layers

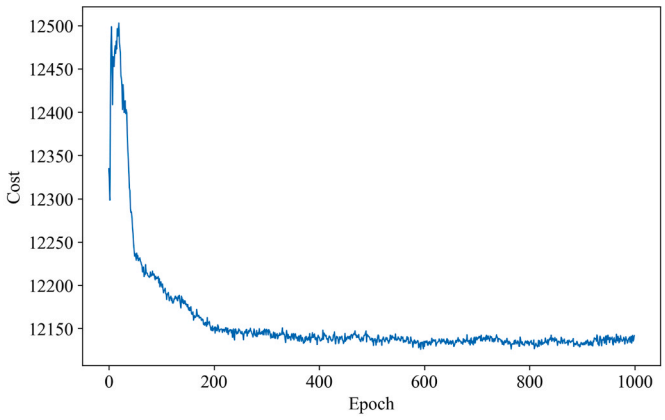


(e) Number of Hidden Units

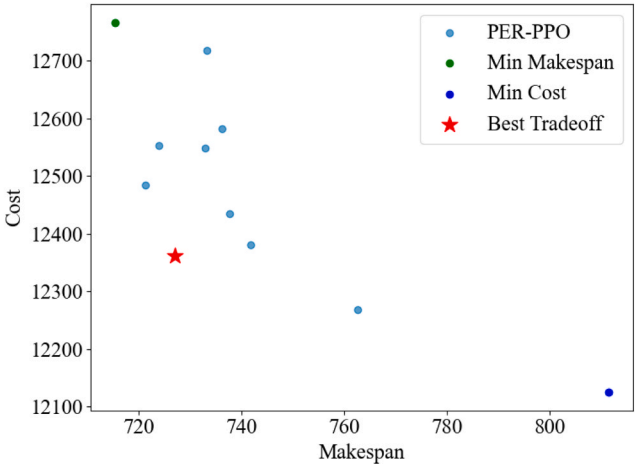
Fig. 9. Training process of E01 under weight setting of (1, 0).



(a) Use makespan as the optimization objective under weight setting of (1, 0)



(b) Use cost as the optimization objectiveunder weight setting of (0, 1)



(c) Solution set

Fig. 10. Training process of the agent and the solution set of K05.

Table 10
Extreme and compromise solutions for different scenarios.

Instance	Extreme solutions				Compromise Solution	
	The optimal cost (Unit)		The optimal makespan (Unit)			
	C_{max}(min)	L_{cost}(RMB)	C_{max}(min)	L_{cost}(RMB)	C_{max}(min)	L_{cost}(RMB)
K01	194.78	1996.28	184.18	2253.04	188.25	2185.53
K02	359.62	6385.86	339.16	6428.23	339.15	6428.22
K03	403.89	5251.36	397.08	5430.04	397.16	5365.32
K04	415.03	6378.00	392.30	6455.24	409.13	6420.27
K05	811.34	12,125.80	715.45	12,765.75	727.10	12,361.70
K06	901.17	15,260.30	928.95	15,520.06	942.05	15,454.85
E01	282.31	2284.45	297.43	2274.96	299.72	2401.45
E02	229.84	3511.07	248.14	3264.30	233.77	3450.37
E03	292.75	5103.04	309.55	4921.37	300.12	5379.88
E04	191.92	5142.09	204.89	4936.79	194.92	5175.51
E05	234.28	8375.58	242.55	8070.39	237.71	8478.69
E06	183.64	7322.92	216.30	6159.10	187.25	7179.88

Table 11
Benchmark rules.

No.	Job rules	Worker rules	No.	Job rules	Worker rules
Rule1	SPT	LWC	Rule5	SPT	HPL
Rule2	MWKR		Rule6	MWKR	
Rule3	MOR		Rule7	MOR	
Rule4	FIFO		Rule8	FIFO	

0.0001 were tested, with 0.0001 yielding the best performance. The number of MLP layers was then optimized. Too many layers may lead to overfitting, while too few limit feature extraction capacity, thus degrading performance. Comparative experiments with 1–4 MLP layers indicated that a two-layer MLP performed best. The NHGNN model was configured with two layers. For each operation node embedding network $\mathrm{MLP}_{\omega_h}(h = 0,\ldots,5)$, the dimensions of the three hidden layers are set to $d_\omega = 64$, and the output layer dimension is set to $d_{out} = 15$.

Table 12
Results of all methods on different cases.

Instance		Proposed	Rule1	Rule2	Rule3	Rule4	Rule5	Rule6	Rule7	Rule8
K01	*GD*	0.0289	0.0557	0.1247	0.0494	0.0703	0.6077	0.0507	0.0447	0.0468
	Spread	0.8999	1	1.1301	1.4112	1.0652	1.1287	1.0875	1.0431	1.1975
	IGD	0.3844	0.7229	0.4235	0.6436	0.5979	0.4009	0.4195	0.6218	0.3844
K02	*GD*	0.0368	0.1739	0.0913	0.0597	0.1674	0.0747	0.0827	0.0551	0.0434
	Spread	0.8092	1	1	1.0721	0.8416	0.9608	1.0584	1.1039	0.855
	IGD	0.2408	0.6843	0.4440	0.4069	0.2816	0.456	0.3717	0.3856	0.2921
K03	*GD*	0.1104	0.0702	0.2906	0.0516	0.1242	0.0384	0.0595	0.0952	0.097
	Spread	0.6238	1	1.0478	0.669	1.0802	1.0205	0.833	1.1929	0.6238
	IGD	0.1712	0.301	0.4964	0.1975	0.3291	0.1735	0.2183	0.2926	0.1712
K04	*GD*	0.0770	0.1873	0.0941	0.1160	0.0893	0.0831	0.1510	0.1513	0.0961
	Spread	0.8231	1.0	1.0300	1.0244	0.8333	1.2987	1.1346	1.0895	0.8310
	IGD	0.1532	0.6201	0.3309	0.2929	0.2675	0.1911	0.2757	0.2903	0.3014
K05	*GD*	0.0452	0.101	0.0811	0.0637	0.0822	0.0899	0.0728	0.0728	0.1658
	Spread	0.7003	1	1	0.9383	0.8914	0.8722	1.1661	1.0722	1.0141
	IGD	0.2858	0.408	0.3693	0.2947	0.3296	0.4488	0.3389	0.3661	0.6434
K06	*GD*	0.0353	0.2374	0.0485	0.0666	0.0419	0.1153	0.1609	0.0499	0.0392
	Spread	0.7169	1	1.075	0.9079	1	0.7054	1.0626	1.0618	0.9533
	IGD	0.1972	0.5897	0.3654	0.3104	0.2516	0.2087	0.2901	0.2729	0.316
E01	*GD*	0.0715	0.1662	0.2269	0.0863	0.0763	0.0347	0.0952	0.0777	0.1888
	Spread	0.2617	0.5803	0.9940	0.9686	0.9981	0.9591	0.5865	0.9949	0.5839
	IGD	0.2818	0.3704	0.5113	0.5406	0.3985	0.7729	0.4934	0.5356	0.3516
E02	*GD*	0.0068	0.4555	0.1166	0.0815	0.0830	0.0130	0.0469	0.0704	0.0521
	Spread	0.1732	0.7566	0.9580	0.9258	0.7341	0.9561	0.9265	0.9598	0.8490
	IGD	0.3230	0.5841	0.4271	0.5970	0.4199	0.7500	0.5758	0.5334	0.4454
E03	*GD*	0.1394	0.2558	0.1018	0.1761	0.0954	0.0156	0.1805	0.2040	0.0935
	Spread	0.2702	0.9430	0.9502	0.9859	0.9454	0.9771	0.9001	0.9868	0.9629
	IGD	0.2141	0.4188	0.4420	0.5468	0.4268	0.6296	0.4596	0.6347	0.4637
E04	*GD*	0.0239	0.3907	0.0990	0.0914	0.1471	0.0129	0.0280	0.0727	0.1082
	Spread	0.1727	0.8670	0.9129	0.9938	0.8407	0.9731	0.8461	0.9599	0.9084
	IGD	0.2462	0.6410	0.4993	0.6912	0.4921	0.8569	0.6398	0.7512	0.5195
E05	*GD*	0.0270	0.6149	0.0759	0.0533	0.1225	0.0741	0.0696	0.0501	0.0443
	Spread	0.3161	0.9246	0.9690	0.9950	0.9727	1.0000	0.8457	0.9569	0.9910
	IGD	0.2122	0.9343	0.5225	0.7329	0.4906	0.7827	0.5254	0.7689	0.4883
E06	*GD*	0.0956	0.5199	0.0692	0.1521	0.1170	0.1418	0.1661	0.1241	0.1189
	Spread	0.3790	0.8073	0.9815	0.9968	0.8938	0.9224	0.7877	0.9884	0.9535
	IGD	0.2194	0.4762	0.3533	0.6612	0.3197	0.7940	0.4178	0.7811	0.3857

Table 13
Significance analysis of the proposed method compared with other methods.

Rule	T-test	Wilcoxon	Significance	T-test	Wilcoxon	Significance	T-test	Wilcoxon	Significance
	GD			*IGD*			*Spread*		
Rule1	0.0035	0.0015	TRUE	0.0001	0.0005	TRUE	0.0000	0.0005	TRUE
Rule2	0.0099	0.0122	TRUE	0.0000	0.0005	TRUE	0.0000	0.0005	TRUE
Rule3	0.0125	0.0210	TRUE	0.0003	0.0005	TRUE	0.0001	0.0005	TRUE
Rule4	0.0161	0.0122	TRUE	0.0001	0.0005	TRUE	0.0002	0.0005	TRUE
Rule5	0.3437	0.3804	FALSE	0.0016	0.0005	TRUE	0.0001	0.0010	TRUE
Rule6	0.0088	0.0161	TRUE	0.0002	0.0005	TRUE	0.0000	0.0005	TRUE
Rule7	0.0021	0.0024	TRUE	0.0003	0.0005	TRUE	0.0000	0.0005	TRUE
Rule8	0.0453	0.0342	TRUE	0.0008	0.0051	TRUE	0.0006	0.0033	TRUE

Both the action selection network MLP_θ and the state value prediction network MLP_ϕ used a dimension of $d_\theta = d_\phi = 64$. To prevent information loss during training, all MLPs had a dropout rate of 0. The number of optimization iterations was set to 1000, with an update interval $E_u = 10$, and the discount factor is $\gamma = 1.0$. The Adam optimizer was used for model training. The total number of experience samples is set to 1024 for the prioritized experience replay buffer. Validation is performed every 1 iterations, and a batch of $B = 10$ instances is replaced every 10 iterations.

5.3.2. Experimental results

The effectiveness of the proposed method was validated across twelve scenarios. Each training iteration consisted of 1000 cycles, using eleven different weight ratios to train the agent. To evaluate the generalization capability, the trained PER-AC-PPO model (trained on E03 and E04) was directly applied to test E05 and E06. Fig. 10(a) shows training results for K05. With prioritized experience replay, the agent learns from high-quality experiences, speeding up policy updates. Initially, when optimizing only makespan, target values fluctuate as the agent adapts to new tasks and adjusts network parameters. After 200 cycles, decision-making improves and the target value stabilizes. Fig. 10 (b) shows similar patterns in cost optimization, with early-stage fluctuations. Results converge after approximately 200 cycles, slower than makespan convergence. Finally, Fig. 10(c) shows the solution set for eleven weight ratios, highlighting the trade-off between cost and completion time. Optimizing one objective often impacts the other, providing decision-makers with a basis for selecting the optimal time-cost balance based on specific scenarios.

Table 10 presents the extreme and compromise solutions obtained across twelve scenarios using the fuzzy membership function. The compromise solutions provide a balanced trade-off between the two, offering decision-makers practical guidance for selecting appropriate scheduling strategies based on scenario requirements. For example, in K06, the compromise solution achieves a makespan of 942.05 min and a

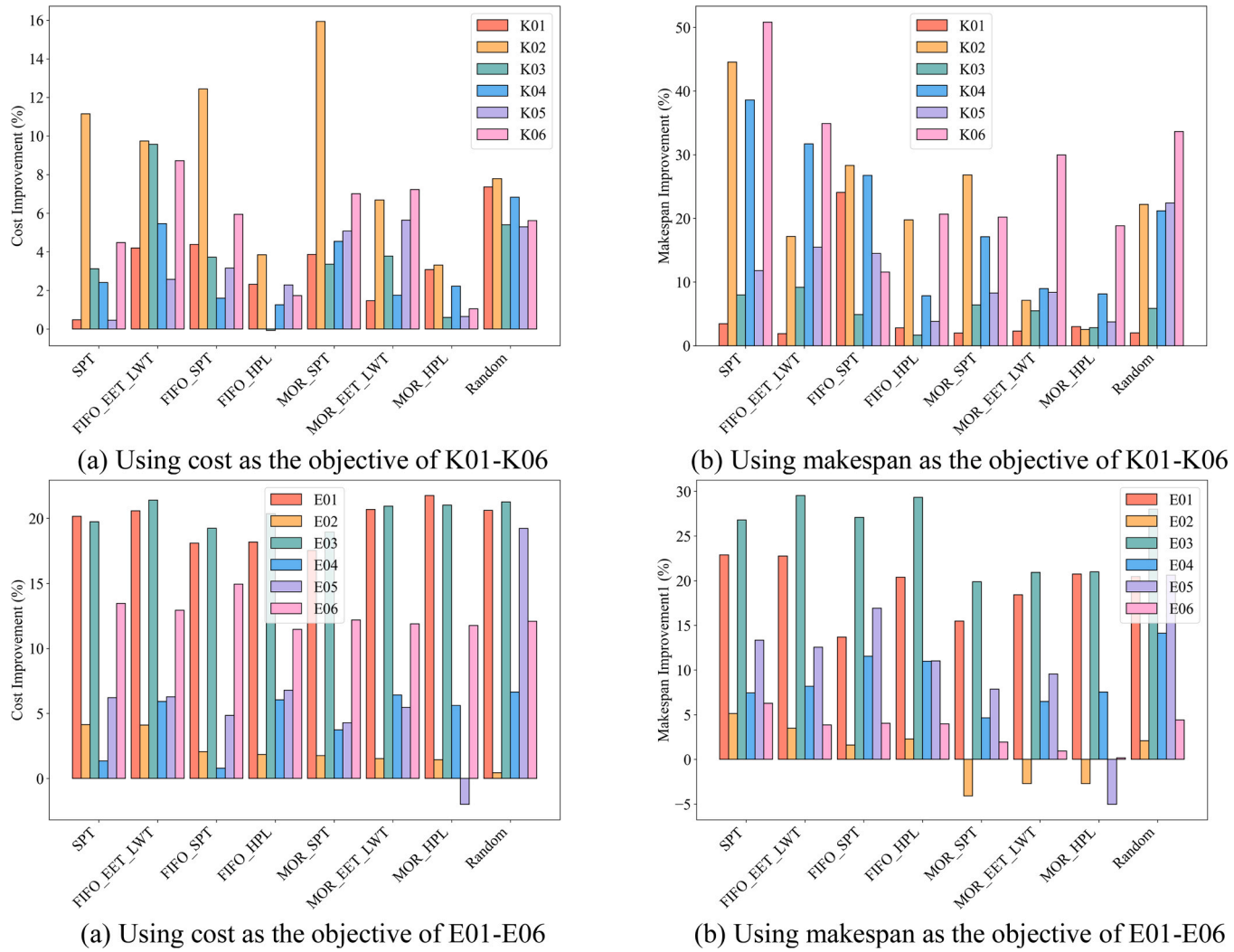


Fig. 11. The percentage increase of each compromise solution relative to PER-AC-PPO across different objectives.

Table 14
Solution results of small-scale instances for all methods.

Instance	PER-AC-PPO		PPO		DDQN		JAYA		ACO	
	C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)
E01	282.31	2274.96	282.31	2274.96	282.31	2274.96	282.31	2274.96	282.31	2274.96
E02	229.84	3264.30	229.84	3264.30	229.84	3264.30	229.84	3264.30	229.84	3264.30

cost of 15,454.85 RMB, balancing the minimum cost (901.17 min) and the shortest makespan (928.95 RMB).

5.3.3. Comparisons with well-known PDR

The performance of the proposed method is assessed for each problem at a specific scale, comparing it with established scheduling rules from the existing literature. Therefore, we selected four well-performing job sequencing rules: shortest processing time (SPT), most work remaining (MWKR), most operations remaining (MOR) and first in first out (FIFO) [50,53]. These rules are combined with the lowest worker cost (LWC) [59] and highest proficiency level (HPL) [60] of the worker assignment rules to form a composite rule, as shown in Table 11, which serves as the benchmark for testing the performance of the proposed DRL model. LWC refers to the selection of workers with the lowest cost per unit time; HPL refers to the selection of the most skilled workers.

For Rules, each instance is run independently 200 times to obtain the best result. The proposed method is repeated 20 times according to different weight ratios W to obtain optimal results as the final experimental results. The experimental results with *GD*, *IGD* and *Spread* on different instances are shown in Table 12. The best values obtained in all algorithms are shown in bold font. The experimental results show that for small-scale instances like E01, some composite rule-based methods such as Rule6 achieve lower *GD* values than PER-AC-PPO, indicating that their solutions are closer to the optimal in single-objective optimization. In contrast, PER-AC-PPO outperforms them in terms of *IGD* and *Spread*, demonstrating superior convergence and diversity of the solution set. As the problem size increases, PER-AC-PPO consistently surpasses all rule-based methods in most instances. It achieves the best values across all three metrics, showing stronger convergence and a more balanced solution distribution. Moreover, the scheduling models trained on E03 and E04 generalize well to large-scale problems, with the model trained on E04 slightly outperforming the one trained on E03.

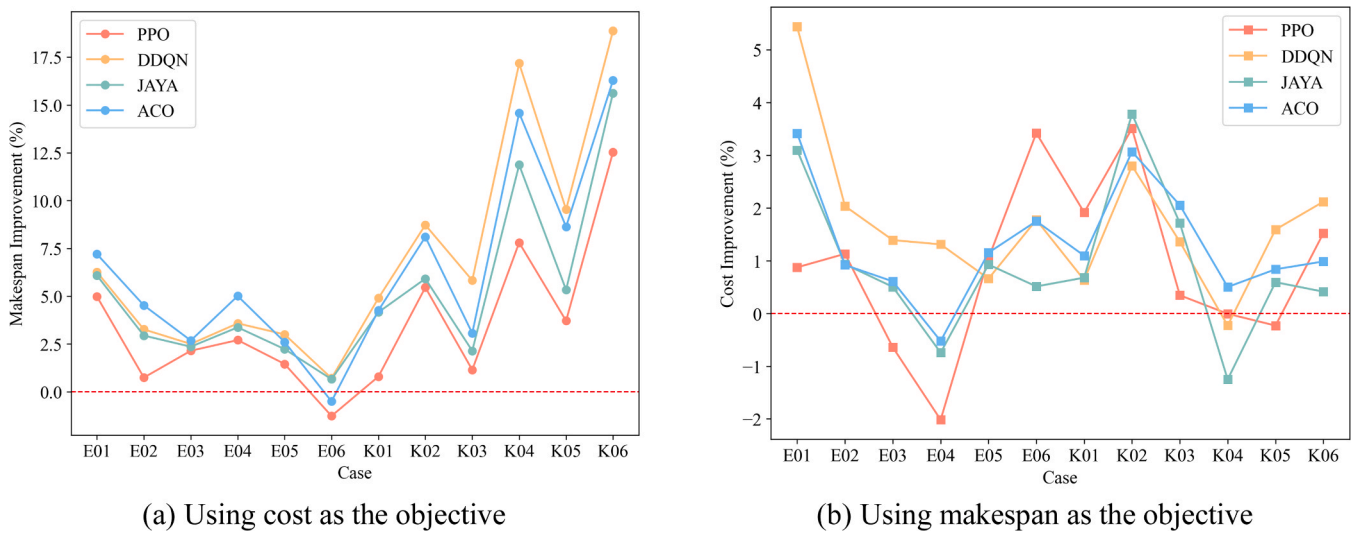


Fig. 12. The percentage increase of each compromise solution relative to PER-AC-PPO across different objectives.

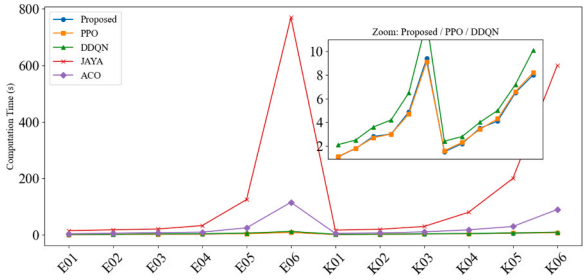


Fig. 13. Comparison of method execution time across different instances.

This confirms the framework's ability for effective knowledge transfer and generalization. Overall, PER-AC-PPO demonstrates robust optimization performance across different problem scales and outperforms traditional composite scheduling rules.

To further validate the effectiveness of the proposed method, statistical significance tests including the *t*-test and Wilcoxon signed-rank test were conducted to compare PER-AC-PPO with rule-based methods across the *GD*, *IGD* and *Spread* metrics. As shown in Table 13, except for Rule5 in the *GD* metric, PER-AC-PPO achieves statistically significant improvements (p-values < 0.05) over all other rule-based methods in most metrics. This confirms the method's consistent and significant superiority across various instances and performance indicators.

As shown in Fig. 11, the optimal compromise solution obtained by

Table 15
Pareto optimal solution *GD*, *IGD* and *Spread* values.

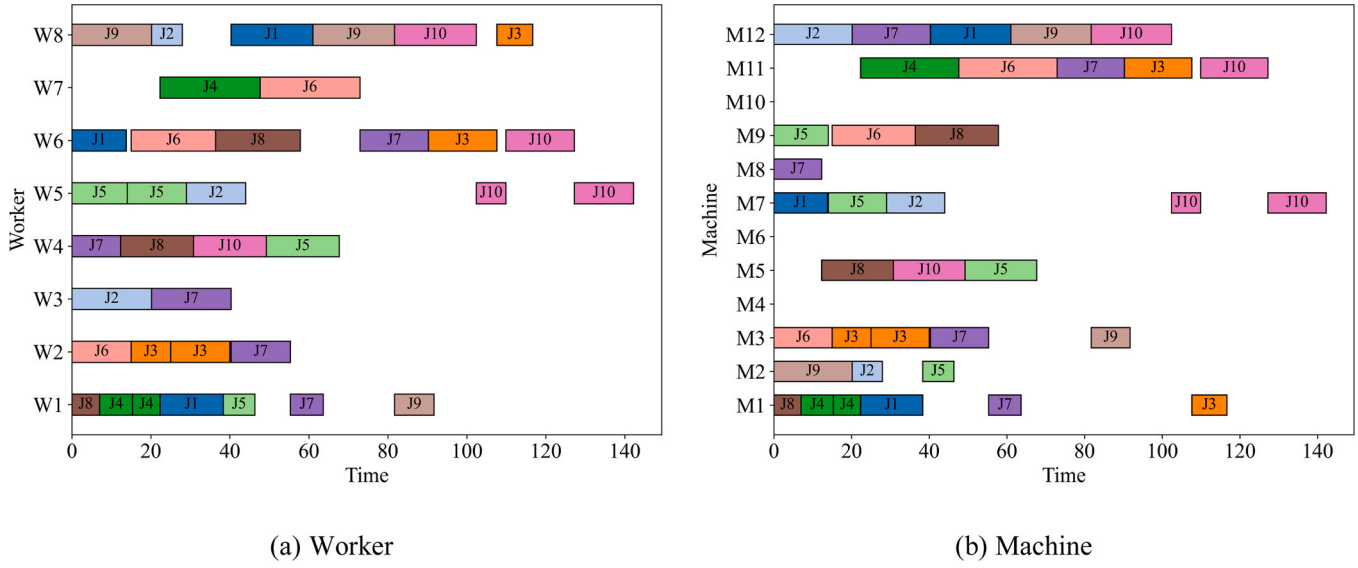
Case		Proposed	PPO	DDQN	JAYA	ACO	Case	Proposed	PPO	DDQN	JAYA	ACO
E01	*GD*	0.0365	0.0641	0.0385	0.1248	0.0545	K01	0.0443	0.0424	0.0473	0.0893	0.0448
	Spread	0.6125	0.6591	0.8313	0.7243	0.6595		0.7614	0.7919	0.6476	0.8471	0.8886
	IGD	0.1065	0.1378	0.1332	0.1253	0.1587		0.0843	0.127	0.0915	0.1499	0.0908
E02	*GD*	0.0245	0.0395	0.0868	0.0427	0.0603	K02	0.0435	0.0778	0.0484	0.0504	0.2460
	Spread	0.8947	0.5924	0.8122	0.8361	0.7546		0.6386	0.6968	0.6847	0.6497	0.8105
	IGD	0.0811	0.109	0.1062	0.0899	0.1497		0.1404	0.1750	0.1561	0.3295	0.1574
E03	*GD*	0.0722	0.0623	0.0313	0.0458	0.0322	K03	0.0435	0.0778	0.0484	0.0504	0.2460
	Spread	0.6543	0.6849	0.7393	0.8552	0.8199		0.6386	0.6968	0.6847	0.6497	0.8105
	IGD	0.0818	0.1177	0.1412	0.1267	0.1482		0.1404	0.1750	0.1561	0.3295	0.1574
E04	*GD*	0.0453	0.0537	0.0484	0.0557	0.089	K04	0.0817	0.0926	0.1007	0.0838	0.1592
	Spread	0.5910	0.7089	0.6733	0.808	0.8823		0.4912	0.6698	0.767	0.4969	0.6478
	IGD	0.1172	0.1821	0.1217	0.2279	0.1318		0.2223	0.2656	0.2843	0.4111	0.2348
E05	*GD*	0.0192	0.0379	0.0343	0.0643	0.1958	K05	0.0444	0.0678	0.1381	0.0686	0.0681
	Spread	0.6272	0.6794	0.9571	0.7572	0.8749		0.5551	0.6619	0.5702	0.5649	0.6732
	IGD	0.0958	0.1446	0.1481	0.0960	0.1096		0.1691	0.2461	0.2261	0.3170	0.1983
E06	*GD*	0.0232	0.0509	0.0327	0.0586	0.0462	K06	0.0357	0.0777	0.2187	0.0513	0.0526
	Spread	0.6198	0.873	0.6899	0.7726	0.9072		0.4845	0.7574	0.6158	0.5615	0.6877
	IGD	0.0695	0.0869	0.0749	0.1115	0.0877		0.1241	0.1681	0.1482	0.1271	0.1683

Table 16
Significance analysis of the proposed method compared with other methods.

Method	T-test	Wilcoxon	Significance	T-test	Wilcoxon	Significance	T-test	Wilcoxon	Significance
	GD			*IGD*			*Spread*		
PPO	0.0006	0.0017	TRUE	0.0000	0.0002	TRUE	0.1136	0.0215	TRUE
DDQN	0.1224	0.0574	FALSE	0.0004	0.0002	TRUE	0.0890	0.1165	FALSE
JAYA	0.0279	0.0266	TRUE	0.0012	0.0002	TRUE	0.0208	0.0327	TRUE
ACO	0.0041	0.0034	TRUE	0.0119	0.0002	TRUE	0.0005	0.0034	TRUE

Table 17
Extreme and compromise solutions of two methods for different scenarios.

Instance	Method	Extreme solutions				Compromise solution	
		The optimal cost (Unit)		The optimal makespan (Unit)			
		C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)	C_{max} (min)	L_{cost} (RMB)
K01	NHGNN-based	194.78	3256.64	184.18	3513.40	184.91	3445.89
	non-NHGNN	217.61	3405.31	190.51	3683.02	196.88	3463.39
K02	NHGNN-based	359.62	6385.86	339.16	6428.23	339.15	6428.22
	non-NHGNN	384.23	6568.10	367.22	6713.38	367.22	6713.38
K03	NHGNN-based	403.89	5251.36	397.08	5430.04	397.16	5365.32
	non-NHGNN	443.66	5465.51	420.53	5734.39	423.04	5471.88
K04	NHGNN-based	415.03	6378.00	392.30	6455.24	409.13	6420.27
	non-NHGNN	436.48	6392.76	418.88	6500.71	420.10	6426.99
K05	NHGNN-based	811.34	12,125.80	715.45	12,765.75	727.10	12,361.70
	non-NHGNN	817.73	12,502.96	784.06	12,750.06	784.06	12,750.06
K06	NHGNN-based	901.17	15,260.30	928.95	15,520.06	942.05	15,454.85
	non-NHGNN	1243.40	15,447.62	1039.38	15,958.00	1065.56	154,86.85
E01	NHGNN-based	282.31	2284.45	297.43	2274.96	299.72	2401.45
	non-NHGNN	291.88	2458.61	298.92	2400.63	307.79	2427.83
E02	NHGNN-based	229.84	3511.07	248.14	3264.30	233.77	3450.37
	non-NHGNN	230.92	3521.52	277.58	3366.25	235.33	3462.81
E03	NHGNN-based	292.75	5103.04	309.55	4921.37	300.12	5379.88
	non-NHGNN	299.26	5619.92	320.75	5005.64	301.27	5416.35
E04	NHGNN-based	191.92	5142.09	204.89	4936.79	194.92	5175.51
	non-NHGNN	194.68	5224.24	211.49	4979.42	195.97	5175.59
E05	NHGNN-based	234.28	8375.58	242.55	8070.39	237.71	8478.69
	non-NHGNN	237.05	8435.06	263.68	8356.06	241.09	8517.42
E06	NHGNN-based	183.64	7322.92	216.30	6159.10	187.25	7179.88
	non-NHGNN	247.21	9826.36	259.97	9115.33	249.38	9658.56



(a) Worker (b) Machine

Fig. 14. Initial scheduling scheme Gantt chart.

PER-AC-PPO serves as a baseline to compare various scheduling rules under different objectives. When minimizing cost (Fig. 11 (a) and (c)), most rules exhibit a significant increase in cost compared to PER-AC-PPO, particularly in large-scale scenarios such as K06, E05, and E06, indicating that PER-AC-PPO has stronger optimization capability in cost control. Similarly, when minimizing the makespan (Fig. 11 (b) and (d)), PER-AC-PPO performs better in reducing the overall makespan. These results indicate that PER-AC-PPO achieves significantly better performance than rule-based methods in both key scheduling objectives, namely cost and makespan, demonstrating strong robustness, convergence, and adaptability in complex and dynamic environments.

5.3.4. Comparison with advanced methods

To confirm that the proposed method is still better than the current dynamic scheduling techniques, two newly suggested real-time scheduling techniques for the DFJSP are re-executed and considered as PPO [49] and DDQN [61], were re-executed and used as baselines for comparison. In addition, two advanced optimization algorithms, the JAYA algorithm [62] and the Ant Colony Optimization (ACO) algorithm [63], were also included. To reduce computational complexity, small-scale instances E01 and E02 were selected to verify the correctness of the constructed Mixed-Integer Programming (MIP) model. Under conditions optimizing makespan and cost separately, Table 14 presents the solution results obtained by different algorithms. The proposed method achieved the best solutions for both makespan and cost on small-scale instances, thereby confirming the correctness and effectiveness of the MIP model developed in this study.

Using the optimal compromise solution obtained by PER-AC-PPO as

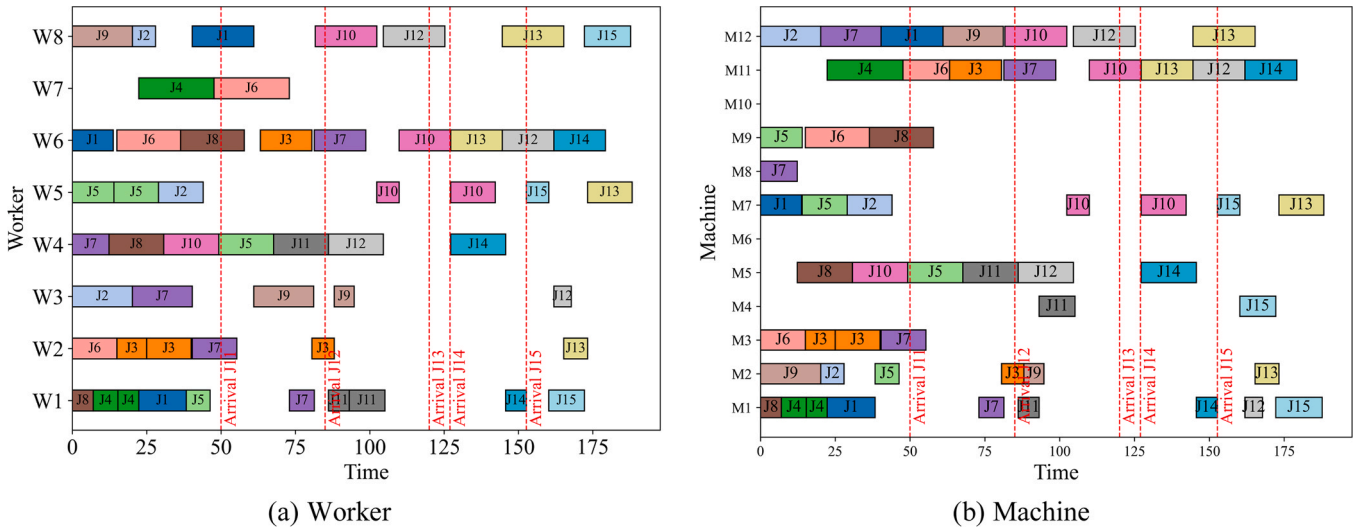


Fig. 15. Gantt chart of the rescheduling scheme after new job arrivals.

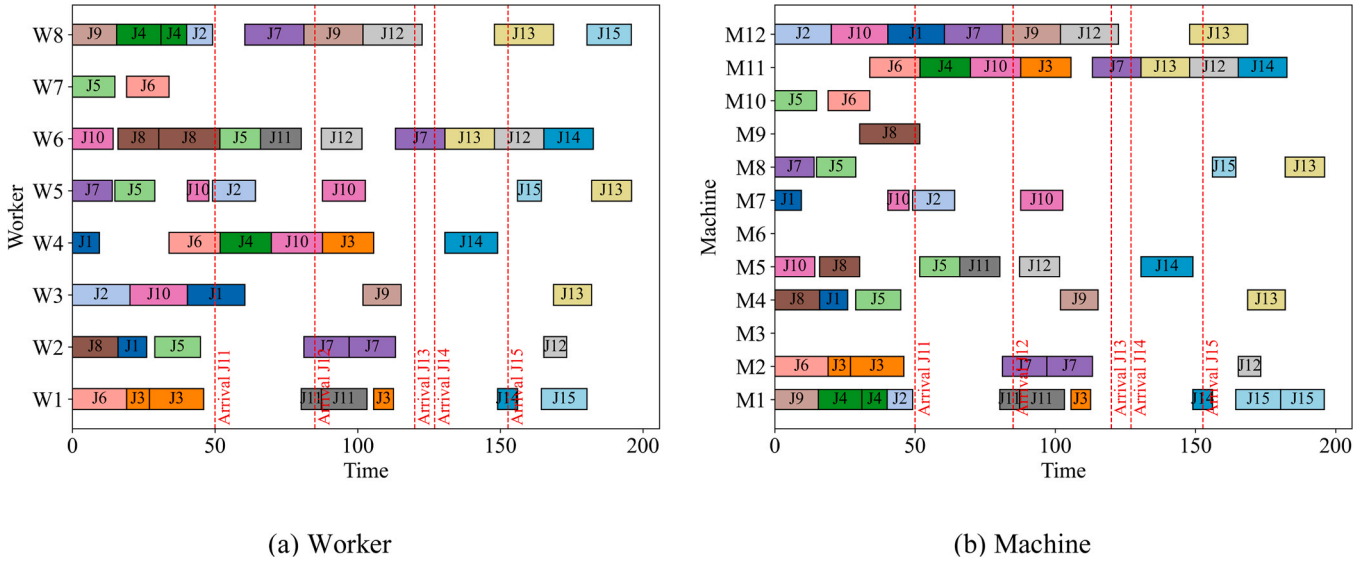


Fig. 16. Gantt chart of the rescheduling scheme obtained by other optimal algorithms.

the baseline, Fig. 12 illustrates the percentage increase of other methods (including PPO, DDQN, JAYA, and ACO) across different cases, reflecting their performance degradation relative to PER-AC-PPO. In terms of makespan optimization (Fig. 12(a)), most algorithms exhibit noticeably higher increase rates compared to PER-AC-PPO, especially in complex cases such as K03, K04, and K06. This indicates that these methods are less effective than PER-AC-PPO in minimizing total completion time. In Fig. 12(b), for the cost optimization objective, DDQN, JAYA, and ACO show significant cost increases in multiple cases, while PER-AC-PPO consistently maintains a lower cost increase rate. Overall, PER-AC-PPO demonstrates superior stability and robustness across both makespan and cost objectives. It outperforms other methods in terms of adaptability, particularly under complex scenarios and multi-objective trade-offs.

As shown in Fig. 13, the comparison of computation time across different instances reveals that the reinforcement learning-based methods (PER-AC-PPO, PPO and DDQN) significantly outperform the heuristic algorithms (JAYA and ACO) in terms of computational efficiency, demonstrating strong real-time performance that is particularly suitable for manufacturing scenarios with dynamic disturbances. Among them, DDQN exhibits slightly higher computation times than Proposed and PPO across all instances, reaching 10.1 s in instance K06. In contrast, PER-AC-PPO and PPO show nearly identical time consumption and consistent trends, indicating similar algorithmic complexity and inference overhead between the two methods.

The experimental results, as shown in Table 15, demonstrate that the proposed method consistently performs well across problems of varying scales. In small-scale instances (including E01 to E02 and K01 to K02), the method achieves the lowest *GD* and *IGD* values, indicating better convergence and closer proximity to the true Pareto front. For medium-scale problems (E03 to E04 and K03 to K04), the method maintains stable performance across all three metrics, exhibiting a balanced trade-off between convergence and solution set diversity. Notably, in large-scale instances (E05 to E06 and K05 to K06), despite the proposed model being trained solely on medium-scale instances E03 and E04 without additional fine-tuning, it still achieves the best results in most evaluation metrics. This highlights the strong generalization capability of the model and its effectiveness in transferring scheduling strategies learned from medium-scale problems to more complex large-scale scenarios. Overall, the proposed method outperforms advanced

optimization algorithms and existing DRL in terms of robustness, scalability, and overall solution quality.

Based on the experimental results in Table 15, a statistical significance analysis was conducted on the PER-AC-PPO and four comparison algorithms (PPO, DDQN, JAYA, and ACO) with respect to three multi-objective optimization metrics. As shown in Table 16, the results indicate that the PER-AC-PPO significantly outperforms PPO, JAYA, and ACO in terms of *GD* and *IGD*, and demonstrates a statistically significant advantage over all comparison methods on the *IGD* metric, highlighting its robustness in solution accuracy and approximation quality. For the *Spread* metric, although the overall differences are less pronounced than those for *IGD*, statistically significant improvements are still observed compared to JAYA and ACO, suggesting that the proposed method also maintains strong competitiveness in solution diversity.

5.3.5. Comparison with feature extraction method without NHGNN

To further validate the superiority of the NHGNN-based feature extraction method, it is compared with a baseline that directly uses raw features for scheduling without NHGNN processing. As shown in Table 17, the NHGNN-based approach consistently outperforms the non-NHGNN method across different task scales and numbers of new jobs. In single-objective optimization, it achieves lower values for both cost and makespan. For compromise solutions, it also delivers superior performance in balancing cost and time, highlighting its strength in multi-objective optimization. Overall, the NHGNN-based method demonstrates robust and stable advantages across all metrics, confirming its ability to enhance solution quality in complex scheduling scenarios.

To demonstrate the impact of new order arrivals on scheduling, a fuzzy membership function method was used to select a compromise solution for the real-world case K01, resulting in a makespan of 142.22 min and a total cost of 1540.42 RMB. The corresponding machine and worker Gantt charts in Fig. 14 visualize the scheduling results, where colors represent different jobs, and operations are arranged sequentially over time.

At time 50 min, a new job arrives, triggering rescheduling. The system first identifies its processing sequence, candidate machines, and workers, then gathers the current resource status and operation progress. Using this as the initial state, the proposed algorithm generates an updated schedule. The same process is repeated as additional jobs arrive at 85.0 min, 120.0 min, 127.0 min, and 152.68 min. The final schedule achieves a makespan of 188.25 min and a total cost of 2185.53 RMB, as illustrated in Fig. 15(a) and (b). The red lines in the figure represent the arrival times of the jobs.

To highlight the superiority of the proposed method in minimizing makespan and cost, the scheduling scheme corresponding to the optimal solutions from the comparison algorithms serves as a benchmark. As shown in Fig. 16, the Gantt charts display the rescheduling outcome after new job arrivals, based on the best-performing results from the other algorithms. This benchmark results in a makespan of 195.99 min and a total cost of 2290.75 RMB, which is higher than the performance achieved by the proposed method.

6. Conclusions and future work

To better reflect real-world manufacturing scenarios, this study extends the DRC-DFJSP by incorporating the impact of worker efficiency and the random arrival of new jobs, resulting in a multi-objective DRC-DFJSP. A real-time scheduling framework is proposed to solve this problem, combining an NHGNN with an improved PPO algorithm, named PER-AC-PPO. The framework simultaneously minimizes makespan and cost, establishing a dynamic scheduling optimization model that comprehensively accounts for worker multi-skills and fatigue factors. To address the optimization model, the problem is formalized as a multi-task MDP, where a linear combination of weight and reward vectors serves as the reward signal. Each weight vector is trained individually, with the corresponding agent parameters stored. A reward function is then designed based on the objective functions to optimize multiple objectives. In capturing the complexity of the workshop status, a three-stage encoding module based on the NHGNN is developed. This module sequentially embeds machine nodes, worker nodes, and operation nodes, allowing for the effective integration of multi-dimensional features in dynamic scheduling. The extracted features are fed into PER-AC-PPO, which is enhanced by an adaptive clipping coefficient mechanism and a prioritized experience replay buffer. This enables the method to achieve optimal objectives under different weight settings and efficiently solve the multi-objective DRC-DFJSP. Through comparative experiments and sensitivity analysis, the effectiveness of the NHGNN embedding strategy and PER-AC-PPO is validated, demonstrating significant performance advantages in large-scale problems, with scalability. Finally, the proposed method is applied to a case study in an aerospace manufacturing workshop. The results show that the proposed method outperforms traditional approaches in production efficiency and scheduling flexibility, demonstrating strong engineering applicability. This work effectively fills the gap in the literature on end-to-end solutions for the DRC-DFJSP.

Although this study integrates three human-related factors (worker constraints, fatigue levels, and multi-skills), there remains substantial potential for further research and exploration in modeling worker characteristics within manufacturing environments. Future work will consider workers' learning and forgetting effects to construct a more comprehensive human profile model. Additionally, the method will be extended to incorporate more manufacturing-related dynamics, such as machine failures and material handling systems, in joint scheduling, thereby further bridging the gap between DRC-DFJSP and actual production and providing stronger theoretical support for practical shop-floor operations.

CRediT authorship contribution statement

Qixiang Cai: Writing – review & editing. **Yiwen Hu:** Writing – original draft, Software, Methodology. **Zequn Zhang:** Writing – review & editing, Supervision. **Jie Chen:** Supervision, Conceptualization. **Dunbing Tang:** Project administration, Conceptualization.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This is a part of research supported by the National Natural Science Foundation of China (92267109, 52305539), the Natural Science Foundation of Jiangsu Province (BK20230880), National Social Science Fund (21BTQ107), the Fundamental Research Funds for the Central Universities (NS2024033), and the Aeronautical Science Foundation of China (2023Z074052007).

Data availability

Derived data supporting the findings of this study are available from the corresponding author Zhang on request.

References

[1] J. Xie, L. Gao, K. Peng, X. Li, L. Haoran, review on flexible job shop scheduling, IET Collab. Intell. Manuf. 1 (2019), https://doi.org/10.1049/iet-cim.2018.0009.
[2] R.T. Nelson, Labor and machine limited production systems, Manag. Sci. 13 (1967) 648–671, https://doi.org/10.1287/mnsc.13.9.648.
[3] J.R. Zamiska, M.Y. Jaber, H.V. Kher, Worker deployment in dual resource constrained systems with a task-type factor, Eur. J. Oper. Res. 177 (2007) 1507–1519, https://doi.org/10.1016/j.ejor.2005.04.018.

[4] B.A. Norman, W. Tharmmaphornphilas, K.L. Needy, B. Bidanda, R.C. Warner, Worker assignment in cellular manufacturing considering technical and human skills, Int. J. Prod. Res. (2002), https://doi.org/10.1080/00207540110118082.

[5] P. Neumann, S. Kihlberg, P. Medbo, S.E. Mathiassen, J. Winkel, A case study evaluating the ergonomic and productivity impacts of partial automation strategies in the electronics industry, Int. J. Prod. Res. 40 (2002) 4059–4075, https://doi.org/10.1080/00207540210148862.

[6] S.R. Kamali, T. Banirostam, H. Motameni, M. Teshnehlab, An immune-based multi-agent system for flexible job shop scheduling problem in dynamic and multi-objective environments, Eng. Appl. Artif. Intell. 123 (2023) 106317, https://doi.org/10.1016/j.engappai.2023.106317.

[7] Y. Zeiträg, J.R. Figueira, N. Horta, R. Neves, Surrogate-assisted automatic evolving of dispatching rules for multi-objective dynamic job shop scheduling using genetic programming, Expert Syst. Appl. 209 (2022) 118194, https://doi.org/10.1016/j.eswa.2022.118194.

[8] C. Zhang, W. Song, Z. Cao, J. Zhang, P.S. Tan, C. Xu, Learning to dispatch for job shop scheduling via deep reinforcement learning. in: Proceedings of the 34th International Conference on Neural Information Processing Systems, Curran Associates Inc, Red Hook, NY, USA, 2020, pp. 1621–1632.

[9] O. Holthaus, C. Rajendran, Efficient dispatching rules for scheduling in a job shop, Int. J. Prod. Econ. 48 (1997) 87–105, https://doi.org/10.1016/S0925-5273(96)00068-0.

[10] P. da Costa, J. Rhuggenaath, Y. Zhang, A. Akcay, U. Kaymak, Learning 2-Opt heuristics for routing problems via deep reinforcement learning, SN Comput. Sci. 2 (2021) 388, https://doi.org/10.1007/s42979-021-00779-2.

[11] K. Lei, P. Guo, Y. Wang, X. Wu, W. Zhao, Solve routing problems with a residual edge-graph attention neural network, Neurocomputing 508 (2022) 79–98, https://doi.org/10.1016/j.neucom.2022.08.005.

[12] L. Yue, K. Peng, L. Ding, J. Mumtaz, L. Lin, T. Zou, Two-stage double deep Q-network algorithm considering external non-dominant set for multi-objective dynamic flexible job shop scheduling problems, Swarm Evolut. Comput. 90 (2024) 101660, https://doi.org/10.1016/j.swevo.2024.101660.

[13] Y. Li, W. Gu, M. Yuan, Y. Tang, Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep q network, Robot. Comput. Integr. Manuf. 74 (2022) 102283, https://doi.org/10.1016/j.rcim.2021.102283.

[14] L. Zhang, Y. Feng, Q. Xiao, Y. Xu, D. Li, D. Yang, Z. Yang, Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times, J. Manuf. Syst. 71 (2023) 257–273, https://doi.org/10.1016/j.jmsy.2023.09.009.

[15] S. Zhang, H. Du, S. Borucki, S. Jin, T. Hou, Z. Li, Dual resource constrained flexible job shop scheduling based on improved quantum genetic algorithm, Machines 9 (2021) 108, https://doi.org/10.3390/machines9060108.

[16] E. Manafi, B. Domenech, R. Tavakkoli-Moghaddam, M. Ranaboldo, A self-learning whale optimization algorithm based on reinforcement learning for a dual-resource flexible job shop scheduling problem, Appl. Soft Comput. 180 (2025) 113436, https://doi.org/10.1016/j.asoc.2025.113436.

[17] P.K.R. Maddikunta, Q.-V. Pham, P. B, N. Deepa, K. Dev, T.R. Gadekallu, R. Ruby, M. Liyanage, Industry 5.0: a survey on enabling technologies and potential applications, J. Ind. Inf. Integr. 26 (2022) 100257, https://doi.org/10.1016/j.jii.2021.100257.

[18] K. Han, W. Gong, Memetic algorithm based on non-dominated levels for flexible job shop scheduling problem with learn-forgetting effect and worker cooperation, Comput. Ind. Eng. 200 (2025) 110845, https://doi.org/10.1016/j.cie.2024.110845.

[19] M. Calzavara, D. Battini, D. Bogataj, F. Sgarbossa, I. Zennaro, Ageing workforce management in manufacturing systems: state of the art and future research agenda, Int. J. Prod. Res. 58 (2020) 729–747, https://doi.org/10.1080/00207543.2019.1600759.

[20] H. Song, J. Li, Z. Du, X. Yu, Y. Xu, Z. Zheng, J. Li, A Q-learning driven multi-objective evolutionary algorithm for worker fatigue dual-resource-constrained distributed hybrid flow shop, Comput. Oper. Res. 175 (2025) 106919, https://doi.org/10.1016/j.cor.2024.106919.

[21] J. a C. Bokhorst, J. Slomp, G. Gaalman, On the who-rule in dual resource constrained (DRC) manufacturing systems, Int. J. Prod. Res. (2004). ⟨https://www.semanticscholar.org/paper/On-the-who-rule-in-Dual-Resource-Constrained-(DRC)-Bokhorst-Slomp/6f1c31e761634b98160679bfdb43ed54962ca114⟩ (accessed August 21, 2024).

[22] Z. Liu, J. Liu, C. Zhuang, F. Wan, Multi-objective complex product assembly scheduling problem considering parallel team and worker skills, J. Manuf. Syst. 63 (2022) 454–470, https://doi.org/10.1016/j.jmsy.2022.05.003.

[23] A. Delgoshaei, M.K.A.M. Ariffin, S. Maleki, Z. Leman, Review evolution of dual-resource-constrained scheduling problems in manufacturing systems: modeling and scheduling methods' trends, Soft Comput. 27 (2023) 18489–18528, https://doi.org/10.1007/s00500-023-09304-4.

[24] F.M. Defersha, D. Obimuyiwa, A.D. Yimer, Mathematical model and simulated annealing algorithm for setup operator constrained flexible job shop scheduling problem, Comput. Ind. Eng. 171 (2022) 108487, https://doi.org/10.1016/j.cie.2022.108487.

[25] X. Wu, J. Peng, X. Xiao, S. Wu, An effective approach for the dual-resource flexible job shop scheduling problem considering loading and unloading, J. Intell. Manuf. 32 (2021) 707–728, https://doi.org/10.1007/s10845-020-01697-5.

[26] L. Meng, C. Zhang, B. Zhang, Y. Ren, Mathematical modeling and optimization of Energy-Conscious flexible job shop scheduling problem with worker flexibility, IEEE Access 7 (2019) 68043–68059, https://doi.org/10.1109/ACCESS.2019.2916468.

[27] H. Li, X. Wu, a survival duration-guided NSGA-III for sustainable flexible job shop scheduling problem considering dual resources, IET Collab. Intell. Manuf. 3 (2021) 119–130, https://doi.org/10.1049/cim2.12003.

[28] W. Tan, X. Yuan, J. Wang, X. Zhang, A fatigue-conscious dual resource constrained flexible job shop scheduling problem by enhanced NSGA-II: an application from casting workshop, Comput. Ind. Eng. 160 (2021) 107557, https://doi.org/10.1016/j.cie.2021.107557.

[29] J. Chang, D. Yu, Y. Hu, W. He, H. Yu, Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival, Processes 10 (2022) 760, https://doi.org/10.3390/pr10040760.

[30] Z. Wang, W. Liao, Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning, J. Intell. Manuf. 35 (2024) 2593–2610, https://doi.org/10.1007/s10845-023-02161-w.

[31] K. Lei, P. Guo, Y. Wang, J. Zhang, X. Meng, L. Qian, Large-Scale dynamic scheduling for flexible Job-Shop with random arrivals of new jobs by hierarchical reinforcement learning, IEEE Trans. Ind. Inform. 20 (2024) 1007–1018, https://doi.org/10.1109/TII.2023.3272661.

[32] M. Zhang, L. Wang, F. Qiu, X. Liu, Dynamic scheduling for flexible job shop with insufficient transportation resources via graph neural network and deep reinforcement learning, Comput. Ind. Eng. 186 (2023) 109718, https://doi.org/10.1016/j.cie.2023.109718.

[33] Z. Liu, H. Mao, G. Sa, H. Liu, J. Tan, Dynamic job-shop scheduling using graph reinforcement learning with auxiliary strategy, J. Manuf. Syst. 73 (2024) 1–18, https://doi.org/10.1016/j.jmsy.2024.01.002.

[34] Y. Zhao, Y. Chen, J. Wu, C. Guo, Dual dynamic attention network for flexible job scheduling with reinforcement learning, Int. Jt. Conf. Neural Netw. (IJCNN) 2024 (2024) 1–8, https://doi.org/10.1109/IJCNN60899.2024.10649988.

[35] D. Huang, H. Zhao, L. Zhang, K. Chen, Learning to dispatch for flexible job shop scheduling based on deep reinforcement learning via graph gated channel transformation, IEEE Access 12 (2024) 50935–50948, https://doi.org/10.1109/ACCESS.2024.3384923.

[36] L. Ding, Z. Guan, M. Rauf, L. Yue, Multi-policy deep reinforcement learning for multi-objective multiplicity flexible job shop scheduling, Swarm Evol. Comput. 87 (2024) 101550, https://doi.org/10.1016/j.swevo.2024.101550.

[37] L. Wan, X. Cui, H. Zhao, L. Fu, C. Li, A novel method for solving dynamic flexible job-shop scheduling problem via DIFFormer and deep reinforcement learning, Comput. Ind. Eng. 198 (2024) 110688, https://doi.org/10.1016/j.cie.2024.110688.

[38] Q. Ji, P. Lan, C. Looney, A probabilistic framework for modeling and real-time monitoring human fatigue, IEEE Trans. Syst. Man Cybern. Part A Syst. Hum. 36 (2006) 862–875, https://doi.org/10.1109/TSMCA.2005.855922.

[39] Y. Soo, M. Nishino, M. Sugi, H. Yokoi, T. Arai, R. Kato, T. Nakamura, J. Ota, Evaluation of frequency band technique in estimating muscle fatigue during dynamic contraction task, IEEE Int. Conf. Robot. Autom. 2009 (2009) 933–938, https://doi.org/10.1109/ROBOT.2009.5152845.

[40] A. Ferjani, A. Ammar, H. Pierreval, S. Elkosantini, A simulation-optimization based heuristic for the online assignment of multi-skilled workers subjected to fatigue in manufacturing systems, Comput. Ind. Eng. 112 (2017) 663–674, https://doi.org/10.1016/j.cie.2017.02.008.

[41] M.Y. Jaber, Z.S. Givi, W.P. Neumann, Incorporating human fatigue and recovery into the learning–forgetting process, Appl. Math. Model. 37 (2013) 7287–7299, https://doi.org/10.1016/j.apm.2013.02.028.

[42] S. Konz, Work/rest: part II – the scientific basis (knowledge base) for the guide1. in: Elsevier Ergonomics Book Series, Elsevier, 2000, pp. 401–427, https://doi.org/10.1016/S1572-347X(00)80033-6.

[43] R. Brauchler, K. Landau, Task analysis: part II—The scientific basis (knowledge base) for the guide 1, Int. J. Ind. Ergon. 22 (1998), https://doi.org/10.1016/S1572-347X(00)80003-8.

[44] C. Zhang, P. Li, Z. Guan, Y. Rao, A tabu search algorithm with a new neighborhood structure for the job shop scheduling problem, Comput. Oper. Res. 34 (2007) 3229–3242, https://doi.org/10.1016/j.cor.2005.12.002.

[45] W. Bożejko, M. Uchroński, M. Wodecki, Parallel hybrid metaheuristics for the flexible job shop problem, Comput. Ind. Eng. 59 (2010) 323–333, https://doi.org/10.1016/j.cie.2010.05.004.

[46] W. Song, X. Chen, Q. Li, Z. Cao, Flexible Job-Shop scheduling via graph neural network and deep reinforcement learning, IEEE Trans. Ind. Inf. 19 (2023) 1600–1610, https://doi.org/10.1109/TII.2022.3189725.

[47] Q. Jiang, J. Wei, Real-time scheduling method for dynamic flexible job shop scheduling, J. Syst. Simul. 36 (07) (2024) 1609–1620, https://doi.org/10.16182/j.issn1004731x.joss.23-0385.

[48] C. Zhang, D. Song, C. Huang, A. Swami, N.V. Chawla, Heterogeneous graph neural network. in: Proceedings of the Twenty Fifth ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, ACM, Anchorage AK USA, 2019, pp. 793–803, https://doi.org/10.1145/3292500.3330961.

[49] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal Policy Optimization Algorithms (2017), https://doi.org/10.48550/arXiv.1707.06347.

[50] P. Brandimarte, Routing and scheduling in a flexible job shop by tabu search, Ann. Oper. Res. 41 (1993) 157–183, https://doi.org/10.1007/BF02023073.

[51] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength pareto approach, IEEE Trans. Evolut. Comput. 3 (1999) 257–271, https://doi.org/10.1109/4235.797969.

[52] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Trans. Evolut. Comput. 6 (2002) 182–197, https://doi.org/10.1109/4235.996017.

[53] S.H. Zanakis, A. Solomon, N. Wishart, S. Dublish, Multi-attribute decision making: a simulation comparison of select methods, Eur. J. Oper. Res. 107 (1998) 507–529, https://doi.org/10.1016/S0377-2217(97)00147-1.
[54] B. Rodrigues De Oliveira, M.A.Q. Duarte, Automatic and Semi-automatic analytic hierarchy process (AHP), TAES (2024) e240009, https://doi.org/10.46420/TAES.e240009.
[55] B. Roy, The outranking approach and the foundations of electre methods, Theor. Decis. 31 (1991) 49–73, https://doi.org/10.1007/BF00134132.
[56] C.T. Chen, C.T. Lin, S.F. Huang, A fuzzy approach for supplier evaluation and selection in supply chain management, Int. J. Prod. Econ. 102 (2006) 289–301, https://doi.org/10.1016/j.ijpe.2005.03.009.
[57] D. Sahoo, P.K. Parida, B. Pati, Efficient fuzzy multi-criteria decision-making for optimal college location selection: a comparative study of min–max fuzzy TOPSIS approach, Results Control Optim. 15 (2024) 100422, https://doi.org/10.1016/j.rico.2024.100422.
[58] J.F. Lima, A. Patiño-León, M. Orellana, J.L. Zambrano-Martinez, Evaluating the impact of membership functions and defuzzification methods in a fuzzy system: case of air quality levels, Appl. Sci. 15 (2025) 1934, https://doi.org/10.3390/app15041934.
[59] M. Montazeri, L.N. Van Wassenhove, Analysis of scheduling rules for an FMS, Int. J. Prod. Res. 28 (1990) 785–802, https://doi.org/10.1080/00207549008942754.
[60] Y. Liu, J. Fan, L. Zhao, W. Shen, C. Zhang, Integration of deep reinforcement learning and multi-agent system for dynamic scheduling of re-entrant hybrid flow shop considering worker fatigue and skill levels, Robot. Comput. Integr. Manuf. 84 (2023) 102605, https://doi.org/10.1016/j.rcim.2023.102605.
[61] H.V. Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double Q-Learning, AAAI Conf. Artif. Intell. (2015), https://doi.org/10.1609/aaai.v30i1.10295.
[62] J. Fan, W. Shen, L. Gao, C. Zhang, Z. Zhang, A hybrid jaya algorithm for solving flexible job shop scheduling problem considering multiple critical paths, J. Manuf. Syst. 60 (2021) 298–311, https://doi.org/10.1016/j.jmsy.2021.05.018.
[63] Z. Jia, J. Yan, J.Y.T. Leung, K. Li, H. Chen, Ant colony optimization algorithm for scheduling jobs with fuzzy processing time on parallel batch machines with different capacities, Appl. Soft Comput. 75 (2019) 548–561, https://doi.org/10.1016/j.asoc.2018.11.027.